A Project Report

On

Improving Sleep Disorder Diagnosis Through Optimized Machine Learning Approaches

Submitted in partial fulfillment for the award of

Bachelor of Technology

Degree In

CSE (Artificial Intelligence and Machine Learning)

By

Vemuri. Lakshmi Niharika(Y22ACM486) **Vemuri. Vyshnavi(Y22ACM487)**

Under the Guidance of

N. Srinivasa Rao Sir

Assistant Professor

And

B. Kishore Babu Sir

Assistant Professor

Department of CSE (Artificial Intelligence and Machine Learning)

Bapatla Engineering College (Autonomous)

(Affiliated to Acharya Nagarjuna University)

BAPATLA – 522 102, Andhra Pradesh, INDIA

2025-2026

Department of

CSE (Artificial Intelligence and Machine Learning)

CERTIFICATE

This is to certify that the Project report entitled **Improving Sleep Disorder Diagnosis Through Optimized Machine Learning Approaches** being submitted by VEMURI. LAKSHMI NIHARIKA (Y22ACM486) And VEMURI. VYSHNAVI (Y22ACM487) in partial fulfillment for the award of the Degree of Bachelor of Technology in CSE (Artificial Intelligence and Machine Learning) to the Acharya Nagarjuna University is a record of Bonafide work carried out by them under our guidance and supervision.

Date:

N. Srinivas Rao	**B. Kishore Babu**	**G. V. Leela Kumari**
Sign of Project Guide	**Sign of Project Guide**	**Sign of Project Coordinator**

K. Mani Deep **Sign of External Examiner**

Sign of H.O.D CSE(AIML) Department

DECLARATION

We hereby declare that the project report entitled **Improving Sleep Disorder Diagnosis Through Optimized Machine Learning Approaches** submitted in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Artificial Intelligence and Machine Learning is a record of our project.

This project has been carried out by us under the guidance of N. Srinivas Rao sir and B. Kishore Babu sir We further declare that this work has not been submitted to any other university or institution for the award of any degree or diploma.

We also declare that all sources of information used in this project have been properly acknowledged.

Vemuri Lakshmi Niharika(Y22ACM486)

Vemuri Vyshnavi (Y22ACM487)

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to all those who have contributed, directly or indirectly, to the successful completion of this project.

We are profoundly grateful to our esteemed project guide, **Mr. N.Srinivas Rao, Assistant Professor, Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning)** and **B. Kishore Babu Assistant Professor, Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning)**, for his invaluable guidance, continuous encouragement, and insightful suggestions throughout the course of this work. His expertise and support have been instrumental in shaping the outcome of this project.

We extend our sincere thanks to **Dr. K. Mani Deep, Head of the Department, Computer Science and Engineering (Artificial Intelligence and Machine Learning)**, for facilitating the resources required for this project.

We are deeply indebted to our respected Principal, **Dr. B. Srinivasa Rao, M.Tech., Ph.D.**, for providing a conducive academic environment and the essential resources required for the successful completion of this work.

We would also like to express our appreciation to our project coordinator, **Mrs. G. V. Leela Kumari, Assistant Professor**, for her valuable guidance and support in the preparation and presentation of this report.

We take this opportunity to acknowledge the support and cooperation extended by all the teaching and non-teaching staff of the department, whose assistance has been invaluable.

We would like to place on record our sincere thanks to our industry mentor, **Ms.N.Naga Chaitanaya Rani**, from **Spypro Security Solutions**, for her professional guidance, practical insights, and continuous support during the internship. Her contribution has significantly enhanced the quality and real-world applicability of this project.

V.L. Niharika (Y22ACM486)

V. Vyshnavi (Y22ACM487)

ABSTRACT

Accurate diagnosis of sleep disorders, including obstructive sleep apnea and insomnia, is critical due to their profound impact on physical and mental health. This study utilizes the Sleep Health and Lifestyle dataset from Kaggle to investigate optimized machine learning approaches for sleep disorder prediction. Feature selection is performed using ANOVA, followed by resampling with SMOTEENN to address class imbalance. Novel engineered features are introduced based on seven classifiers: Random Forest, Gradient Boosting, Gaussian Naive Bayes, K-Nearest Neighbors, Decision Tree, Logistic Regression, and Support Vector Machine, generating a total of seven additional features. Multiple machine learning algorithms, including Logistic Regression, Random Forest, Decision Tree, SVM, Extra Trees, Extreme Gradient Boosting, Gradient Boosting, LightGBM, CatBoost, AdaBoost, Naive Bayes, Voting Classifier, and Stacking Classifier are applied. High predictive performance is achieved using the Voting Classifier with engineered features, reaching 97.3% accuracy for both five- and two-feature sets. Using original features, Stacking Classifier attains 88.0% accuracy with five features and 80.0% with two features. Explainable AI techniques, LIME and SHAP, are incorporated to interpret model predictions, enhancing transparency in clinical decision support.

KEY WORDS: Sleep Disorder, Machine Learning, Feature Engineering, SMOTEENN, Classification, Ensemble Learning, Explainable AI.

TABLE OF CONTENTS

SNO	CONTENTS	PAGE.NO
	CERTIFICATE	ii
	DECLARATION	iii
	ACKNOWLEDGEMENT	iv
	ABSTRACT	v
	LIST OF TABLES	viii
	LIST OF FIGURES	viii
	LIST OF ABBREVATIONS	ix
1	INTRODUCTION	1
	1.1. Introduction	1
	1.2.Problem Statement	1
	1.3.Scope of the Project	2
	1.4.Objectives	3
2	FEASIBILITY STUDY	4
3	LITERATURE SURVEY	5
	3.1.Literature Survey	5
	3.2.Existing System	6
	3.2.1. Disadvantages of Existing System	7
	3.3. Proposed System	7
	3.3.1. Advantages of Proposed System	8
4	SYSTEM ANALYSIS	9
	4.1. Functional Requirements	9
	4.2. Non-Functional Requirements	9
	4.3. System Requirements	10

4.3.1. Software Requirements……10

4.3.2. Hardware Requirements……10

4.4. System Environment……11

5 SYSTEM DESIGN……12

5.1. System Architecture……12

5.2. Data Flow Diagram……13

5.3. UML Diagrams……16

5.4. Control Flow Diagram……22

5.5. Database Design……24

6 METHODOLOGY……26

6.1. Technologies Used……26

6.2. Methodology Overview……27

6.3. Module Description……29

7 IMPLEMENTATION……32

7.1. Modules……32

7.2. Algorithms……34

7.3. Source Code……35

8 SYSTEM TESTING……56

8.1. Types of Testing……56

8.2. Test Cases……57

9 RESULTS……59

10 CONCLUSION AND FUTURE SCOPE……66

11 REFERENCES……68

LIST OF TABLES

Table No	Table Name	Page.No
8.2.1	Test Cases	58
9.1.1	Performance Evaluation – Engineered 5 Features	59
9.1.2	Performance Evaluation – Engineered 2 Features	60

LIST OF FIGURES

Figure. No	Figure Name	Page.No
5.1	System Architecture	12
5.2	Data Flow Diagram	15
5.3.1	Use Case Diagram	17
5.3.2	Class Diagram	18
5.3.3	Activity Diagram	19
5.3.4	Sequence Diagram	20
5.3.5	Collaboration Diagram	21
5.3.6	Component Diagram	21
5.3.7	Deployment Diagram	22
5.4	Control Flow Diagram	23
5.5	Database Design	25
6.1	Technologies Used	27
6.2.1	Methodology Overview Diagram	28
6.2.2	Design Methodology	29
9.2.1	Performance Comparison of Classification Algorithms	61
9.3.1	Homepage Interface of Sleep Disorder Prediction System	63
9.3.2	Account Creation Module for Sleep Disorder Prediction Application	63
9.3.3	User Sign-In Interface for System Access	63
9.3.4	Optimized Five-Feature Engineered Model Output Sleep Disorder Diagnosis	64
9.3.5	Reduced Feature (2-Feature) Model Performance Output of Sleep Disorder Diagnosis	65

LIST OF ABBREVATIONS

AI – Artificial Intelligence

ML – Machine Learning

DL – Deep Learning

EDA – Exploratory Data Analysis

ANOVA – Analysis of Variance

SVM – Support Vector Machine

KNN – K-Nearest Neighbors

RF – Random Forest

DT – Decision Tree

LR – Logistic Regression

XGB – Extreme Gradient Boosting (XGBoost)

LGBM – Light Gradient Boosting Machine (LightGBM)

CB – CatBoost

SMOTE – Synthetic Minority Over-sampling Technique

SMOTEENN – SMOTE with Edited Nearest Neighbors

API – Application Programming Interface

UI – User Interface

SQL – Structured Query Language

LIME – Local Interpretable Model-Agnostic Explanations

SHAP – SHapley Additive exPlanations

PSG – Polysomnography

BMI – Body Mass Index

CHAPTER-1: INTRODUCTION

1.1. Introduction

Sleep plays a fundamental role in maintaining both physical and mental health by enabling the body to heal, regulate vital processes, and support optimal cognitive function. Adequate sleep is essential for individuals of all age groups, especially vulnerable populations such as children and older adults. Short-term sleep deprivation can lead to reduced concentration, impaired memory, and increased risk of accidents, while chronic lack of sleep contributes to serious health conditions including cardiovascular disease, diabetes, depression, and obesity. Understanding and managing sleep disorders is therefore a critical healthcare priority.

Sleep quality is typically assessed by analyzing various sleep stages: Wakefulness, N1, N2, N3, and REM. Each stage is associated with distinct brain wave patterns and physiological responses. Wakefulness involves active brain activity, while N1 and N2 represent progressively deeper stages of light sleep. N3 is the deepest non-REM stage, crucial for physical recovery, and REM sleep involves vivid dreams and brain activity similar to waking states. Accurate classification of these stages provides valuable insights into sleep health and helps in diagnosing disorders such as insomnia and sleep apnea.

Polysomnography (PSG) is commonly used for this analysis but is resource-intensive and subject to manual interpretation errors. As a result, automated approaches using data-driven techniques have become vital for improving sleep disorder detection and diagnosis.

1.2. Problem Statement

- Sleep disorders arise from unhealthy lifestyles, stress, irregular sleep patterns, and medical conditions, leading to difficulties in diagnosing them effectively using conventional manual approaches that are often time-consuming, subjective, and prone to misinterpretation or human error.
- Traditional diagnosis methods rely heavily on manual inspection of sleep stages and polysomnography data, which require specialized expertise, significant time, and costly resources, limiting scalability and making timely, accurate sleep disorder detection challenging across larger populations.

- Individuals suffering from sleep disorders, including insomnia and sleep apnea, along with healthcare professionals, are significantly impacted as they struggle with delayed diagnosis, misinterpretations, poor health outcomes, and insufficient treatment guidance due to limitations in existing diagnostic techniques.
- Beyond individual health, untreated sleep disorders contribute to reduced productivity, increased workplace accidents, higher healthcare expenses, and escalated risks of cardiovascular, neurological, and psychological issues, creating both social and economic burdens that worsen without accurate diagnostic support systems.
- To address these challenges, this work proposes an optimized machine learning-based diagnostic framework using advanced feature engineering, ANOVA feature selection, SMOTEENN resampling, and multiple classifiers, along with Explainable AI methods, to ensure accurate, transparent, and scalable sleep disorder prediction.

1.3. Scope of the Project

The scope of this project focuses on developing an intelligent and efficient system for the prediction and diagnosis of sleep disorders using optimized machine learning techniques. The system utilizes health and lifestyle-related data to identify common sleep disorders such as insomnia and sleep apnea, enabling early detection and improved healthcare support.

This project covers the complete workflow starting from data collection and preprocessing to model training, evaluation, and deployment. Advanced techniques such as ANOVA-based feature selection, SMOTEENN resampling for handling class imbalance, and feature engineering are incorporated to enhance model performance and accuracy. Multiple machine learning algorithms, including ensemble methods like Voting and Stacking classifiers, are applied to achieve reliable predictions.

Additionally, the project integrates Explainable AI techniques such as LIME and SHAP to provide transparency and interpretability of model decisions, making the system more trustworthy for healthcare applications. A user-friendly web interface using Flask is also included, allowing users to input their health data and receive real-time predictions.

The system is designed to be scalable, efficient, and applicable in real-world scenarios such as clinical decision support systems and personal health monitoring. However, the project is limited

to dataset-based predictions and does not include real-time physiological data like EEG or PSG signals.

1.4. Objectives

The objective of this work is to develop an optimized machine learning-based system for accurate and reliable prediction of sleep disorders using the Sleep Health and Lifestyle dataset. The approach focuses on improving diagnostic performance through ANOVA-based feature selection, SMOTEENN resampling to address class imbalance, and advanced feature engineering using multiple classifiers, including Random Forest, Gradient Boosting, Gaussian Naive Bayes, K-Nearest Neighbors, Decision Tree, Logistic Regression, and Support Vector Machine. A variety of algorithms such as Logistic Regression, Random Forest, Decision Tree, SVM, Extra Trees, Extreme Gradient Boosting, Gradient Boosting, LightGBM, CatBoost, AdaBoost, Voting Classifier, and Stacking Classifier are applied. Furthermore, the work extends by incorporating Explainable AI techniques, LIME and SHAP, to provide transparency and interpretability in model decision-making.

CHAPTER-2: FEASIBILITY STUDY

A feasibility study is an analytical tool used to evaluate the practicality of a proposed project or business idea. It assesses various factors such as financial viability, technical requirements, legal constraints, and market demand.

Types of feasibility studies:

There are different types of feasibility studies that each focus on a unique aspect of projects and project planning. By understanding the nuances of each, you'll become better equipped to make well-informed decisions, mitigate risks, and ultimately steer your project toward success.

Technical feasibility: Technical feasibility digs into the nuts and bolts of the project. You're looking at what kind of technology you'll need, whether it's available, and if it can be integrated into your current systems. It's like checking if you have all the ingredients you need before cooking a specific recipe.

Economic feasibility: This study is all about the money — how much the project will cost and what kinds of economic or profitability benefits it will bring forth. With an economic feasibility study, you're most often doing a cost-benefit analysis to see if the financials add up in your favor. It's like weighing the pros and cons but in dollar signs.

Operational feasibility: An operational feasibility study will help you see how the project fits into your current operations and operational goals and resources. After completing this type of study, you should know if your project will require new workflows and if your team can handle project tasks alongside their current workloads.

This study also evaluates whether the organization has the expertise to accomplish all project goals.

CHAPTER-3: LITERATURE SURVEY

3.1. Literature Survey

1.Optimizing Sleep Disorder Classification Using Bayesian Optimization:

The Gradient Boosting Classifiers enhance the classification of sleep problems in this research. The data is pre-treated to guarantee quality and the aspects to be addressed include health and lifestyle. The Bayesian Optimization increases the performance and optimizes model parameters.

Optimization enhances the performance of the models and the results of the diagnostic process since the accuracy increased to 92.92 to 94.44.

2.Advanced Neural Networks for Sleep Disorder Classification

An AD-ANN is developed by means of CNN, RNN, and ensemble. The model is meant to process spatial and temporal sleep data. Standard datasets that are trained include Sleep-EDF and PhysioNet. The model achieved 98.2 percent accuracy, precision, and recall in the stages of sleep. The method allows the diagnosis of sleep disorder on a scale and at an accurate rate.

3.Revolutionizing Sleep Disorder Diagnosis: A Multi-Task Learning Approach

The proposed paper proposes a CNN-based multi-task learning model to enhance the diagnosis of sleep problems. Rather than using complex PSG, it uses EEG signals to save both time and money. The model uses STFT and CWT time-frequency images.

It enhances the accuracy by integrating Genetic Algorithm and Q-learning forecasts. The 98% precision of the model demonstrates its effectiveness and accuracy to identify sleep problems with the help of the automated system.

4. From Narratives to Diagnosis: SleepCare Platform

SleepCare, a ML application, categorizes sleep issues by utilizing unstructured narrative data in the form of physician notes, online forums and telehealth reports. In the elderly, sleep problems are usual, and these problems may reduce cognitive abilities and life quality. Other diagnostic methods are required since polysomnography is costly and not affordable.

The given strategy categorizes textual descriptions of sleep into clinical outcomes: stress-related, neurodegenerative, and respiratory disorders with the help of the NLP. It is a three-step classification pipeline and includes Multinomial Naïve Bayes, SVM with word embeddings and transformer-based BERT model to take into account syntactic and contextual information.

The model was tested with 475 tagged sleep narratives with the help of cross-validation. The SVM based BERT-based model was 81% accurate and had equal performance across classes.

- Recent advancements in sleep disorder diagnosis employ diverse machine learning and deep learning frameworks, ranging from optimization-driven classifiers to hybrid neural networks and decision-support models. Bayesian-optimized Gradient Boosting improved accuracy but faced computational overhead.
- Advanced deep neural networks such as AD-ANN achieved remarkable precision on PSG datasets, though resource requirements limited scalability. NLP-based frameworks like sleepCare enabled narrative-driven classification but were constrained by small datasets and overlapping symptom categories.
- Neutrosophic machine learning offered robustness in handling uncertainty, yet adoption barriers remain due to interpretability and reliance on detailed clinical data.
- Despite high reported accuracies, most models suffer from limited dataset sizes, lack of generalizability, and real-world validation gaps, highlighting the need for scalable, explainable, and clinically adaptable solutions.

3.2. Existing System

Existing systems for sleep disorder diagnosis have explored various machine learning and deep learning techniques to automate the classification of sleep stages and detection of disorders. These approaches often begin with data preprocessing and feature selection to improve model efficiency by reducing irrelevant or redundant features. Some systems utilize clustering algorithms such as MCMST to group data patterns before applying feature projection techniques like PCA, LDA, SVD, t-SNE, NCA, Isomap, and Polynomial Regression. Others employ hybrid models that integrate both data-driven intelligence and domain knowledge, combining inputs from EEG and EOG channels with architectures such as Temporal Fully Convolutional Networks and multi-task feature mapping. Machine learning methods such as K-Nearest Neighbors (KNN), Random Forest,

and Artificial Neural Networks (ANN) have been widely used, often optimized through techniques like genetic algorithms. While these systems demonstrate improvements over manual methods, challenges remain in achieving high accuracy, generalization across datasets, and interpretability in clinical settings.

3.2.1. Disadvantages of Existing System

- Existing systems often rely heavily on EEG and EOG data, making them less applicable for datasets focused on lifestyle and health indicators without specialized medical sensor-based inputs.
- Many approaches use dimensionality reduction like PCA and LDA, which sometimes discard important information, leading to reduced interpretability and lower predictive power when applied to diverse and imbalanced sleep-related data.
- Hybrid models integrating multiple modalities, while powerful, become computationally expensive and complex, limiting their adaptability to lightweight frameworks and practical deployment scenarios requiring efficiency and real-time predictions.
- Existing systems lack robust mechanisms for explainability, making it difficult to interpret decisions in clinical contexts, which reduces trust and limits their widespread adoption in medical diagnosis of sleep disorders.

3.3. Proposed System

The proposed system aims to enhance the diagnosis of sleep disorders by leveraging optimized machine learning techniques on the Sleep Health and Lifestyle dataset from Kaggle. The workflow begins with data preprocessing, where missing values are handled, categorical variables are encoded, and numerical features are standardized. Feature selection is conducted using ANOVA to identify the most significant predictors, while SMOTEENN is applied to address class imbalance and improve classification reliability. Feature engineering is introduced by creating seven new features derived from models including Random Forest, Gradient Boosting, Gaussian Naive Bayes, K-Nearest Neighbors, Decision Tree, Logistic Regression, and Support Vector Machine, thereby enriching the feature space. A range of classifiers is employed for training, including Logistic Regression, Random Forest, Decision Tree, Support Vector Machine, Extra Trees, Extreme Gradient Boosting, Gradient Boosting, LightGBM, CatBoost, AdaBoost, Naive Bayes,

Voting Classifier, and Stacking Classifier. The system is designed to provide accurate and robust sleep disorder predictions.

3.3.1. Advantages of Proposed System

- The proposed system integrates advanced preprocessing steps like SMOTEENN and ANOVA feature selection, ensuring balanced, clean, and highly informative datasets for reliable predictions across diverse health and lifestyle indicators.
- Feature engineering creates seven additional predictors derived from strong classifiers, enriching the feature space and capturing complex relationships between variables, significantly improving the depth and quality of input data.
- A comprehensive set of classifiers including Random Forest, Gradient Boosting, LightGBM, CatBoost, and ensemble methods ensures high accuracy, robustness, and adaptability across varying feature interactions and data distributions.
- The system is designed for practical usability, employing Flask for user interaction and explainable methods for interpretation, bridging technical prediction power with accessibility and transparency for end-users and clinical relevance.

CHAPTER-4: SYSTEM ANALYSIS

4.1. Functional Requirements

The functional requirements of the proposed system define the core operations performed at each stage of the sleep disorder prediction process. The system includes the following functionalities:

1. **Data Collection**: The system shall collect the Sleep Health and Lifestyle dataset from reliable sources. It shall store and manage the dataset for further processing and analysis.
2. **Data Preprocessing**: The system shall preprocess the collected data by handling missing values, removing inconsistencies, encoding categorical variables, and scaling numerical features to prepare the dataset for model training.
3. **Training and Testing:** The system shall divide the dataset into training and testing sets. It shall use the training data to train machine learning models and the testing data to evaluate their performance.
4. **Modelling:** The system shall build and implement multiple machine learning models such as Logistic Regression, Decision Tree, Random Forest, Support Vector Machine, and ensemble models to improve prediction accuracy.
5. **Prediction:** The system shall accept input data from the user and generate predictions regarding sleep disorders using the trained model. The results shall be displayed clearly to the user.

4.2. Non- Functional Requirements

1. Reliability: The system must ensure consistent performance under varying input conditions, maintaining stable predictions without failures, ensuring dependability for medical practitioners relying on accurate outcomes for timely diagnosis and decision-making in critical healthcare situations.

2. Scalability: The framework should efficiently scale to handle larger datasets, additional patient records, or extended clinical attributes without performance degradation, ensuring adaptability for future integration with hospital databases and broader medical applications.

3. Security: Patient data confidentiality must be preserved by implementing strong data protection measures, secure storage, and restricted access, ensuring compliance with healthcare data privacy regulations and safeguarding sensitive health records from potential breaches.

4. Usability: The system should provide an intuitive and user-friendly interface, enabling healthcare professionals with varying technical expertise to input patient features, interpret outputs, and integrate predictions into clinical workflows without additional complexity.

5. Maintainability: The architecture must support easy updates, bug fixes, and model retraining to accommodate new medical knowledge, evolving datasets, and changing healthcare requirements while ensuring minimal downtime and simplified system enhancements.

4.3. System Requirements

System requirements specify the hardware and software needed to develop and run the Sleep Disorder Prediction System effectively**.**

4.3.1. Software Requirements

1) Software: Anaconda (2023.03-1)
2) Primary Language: Python (3.10.9)
3) Web Framework: Flask
4) Development Environment: Jupyter Notebook
5) Database: Sqlite3
6) Front-End Technologies: HTML, CSS, JavaScript and Bootstrap4

4.3.2. Hardware Requirements

1) Operating System: Windows 10 (Min) and Above
2) Processor: i5 9th Gen and Above
3) RAM: 8GB (Min) – 16 or 32 Preferable
4) Hard Disk: Free space 100 GB (Min)

4.4. System Environment

The system environment defines the set of tools, technologies, and platforms used for the development and deployment of the sleep disorder prediction system. The project is implemented using Python as the primary programming language due to its simplicity, flexibility, and strong support for machine learning applications.

Various libraries are utilized to handle different stages of the workflow. NumPy and Pandas are used for efficient data manipulation and preprocessing, while Scikit-learn is applied for implementing machine learning algorithms and evaluation techniques. For data visualization and analysis, Matplotlib and Seaborn are used. Advanced machine learning models are developed using libraries such as XGBoost, LightGBM, and CatBoost, which enhance predictive performance and efficiency.

The development environment includes Anaconda, which provides package management and a stable ecosystem for data science applications. Coding and experimentation are carried out using Jupyter Notebook, allowing interactive execution and visualization of results. For deployment, the Flask web framework is used to create a user-friendly interface where users can input health-related data and obtain real-time predictions.

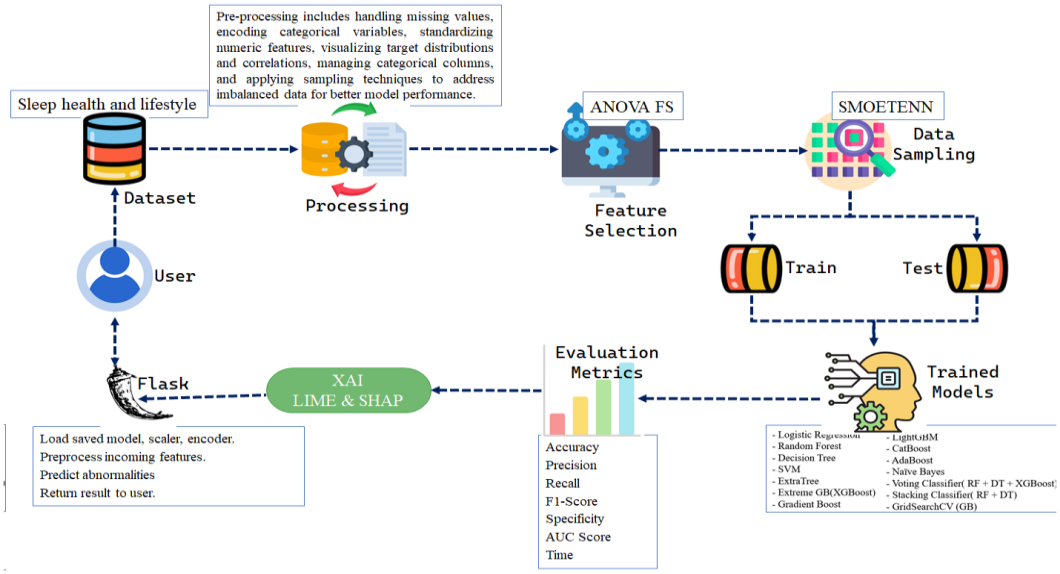
The system operates on a Windows operating system with standard hardware requirements, ensuring smooth performance and accessibility. This environment supports efficient model training, testing, and deployment, making the system suitable for practical applications in healthcare prediction.

CHAPTER-5: SYSTEM DESIGN

System Design describes the overall architecture, components, data flow, and interaction between different modules of the system. It provides a blueprint for implementing the system efficiently and ensures that all functional and non-functional requirements are satisfied

5.1. System Architecture

The system architecture defines the overall structure of the Sleep Disorder Prediction System, showing how different components interact with each other. The system follows a layered architecture, ensuring modularity, scalability, and efficient processing.



Architecture Components

1. Input Layer

- Accepts user input such as age, gender, BMI, sleep duration, stress level, etc.
- Data is collected through a web interface (Flask application)

2. Data Processing Layer

- Handles data preprocessing
- Includes cleaning, encoding, scaling, and transformation
- Ensures data consistency before model processing

3. Model Layer

- Contains trained machine learning models
- Includes algorithms like Random Forest, SVM, XGBoost, etc.

- Performs prediction based on processed input

4. Output Layer

- Displays prediction results
- Provides user-friendly output (e.g., type of sleep disorder)

5.2. Data Flow Diagram

A Data Flow Diagram (DFD) is a graphical representation that illustrates how information moves through a system, showing the flow of data between different processes and data stores. In the context of sleep disorder prediction, a DFD provides a clear picture of how user inputs such as lifestyle and health-related details are processed by the system to generate meaningful outputs. It helps visualize the way raw inputs are transformed step by step into predictions about potential sleep abnormalities.

The DFD represents processes like data preprocessing, feature transformation, and classification in an abstract manner, highlighting the logical flow rather than technical details. For instance, user-provided attributes are first processed, then passed through feature extraction and transformation stages, and finally analyzed to produce predictions. Additionally, the DFD includes interactions where processed data is stored or retrieved for further use, ensuring smooth data handling across different stages of the workflow.

By focusing on inputs, processes, and outputs, the DFD enables a simplified understanding of system functionality. It makes it easier to communicate how the system processes health-related information, ensuring clarity and transparency in how predictions are generated, ultimately supporting accurate and efficient diagnosis of sleep disorders

Goals of DFD: To visually represent how user-provided lifestyle and health data flows through various processes, ensuring a clear understanding of transformations leading to sleep disorder predictions without requiring technical complexity for interpretation.

To simplify the identification of processes such as preprocessing, feature handling, and classification, making it easier to observe how each stage contributes to generating reliable and accurate outputs in sleep disorder prediction.

To enhance transparency by showing logical data movement within the system, ensuring that all necessary inputs and outputs are clearly defined for smooth information flow, thereby improving prediction reliability and diagnostic accuracy.

To provide a clear system-level view of data interaction and storage, ensuring efficient tracking of how information is handled throughout the workflow, which supports trust and consistency in sleep disorder prediction tasks. To facilitate better communication of system functionality by abstractly modeling how input data is converted into predictive outcomes, supporting healthcare-focused applications by ensuring clarity in the processes behind accurate disorder detection.

DFD Levels

Level 0 (Context Diagram)

- User → Inputs data
- System → Processes data
- Output → Prediction result

Level 1

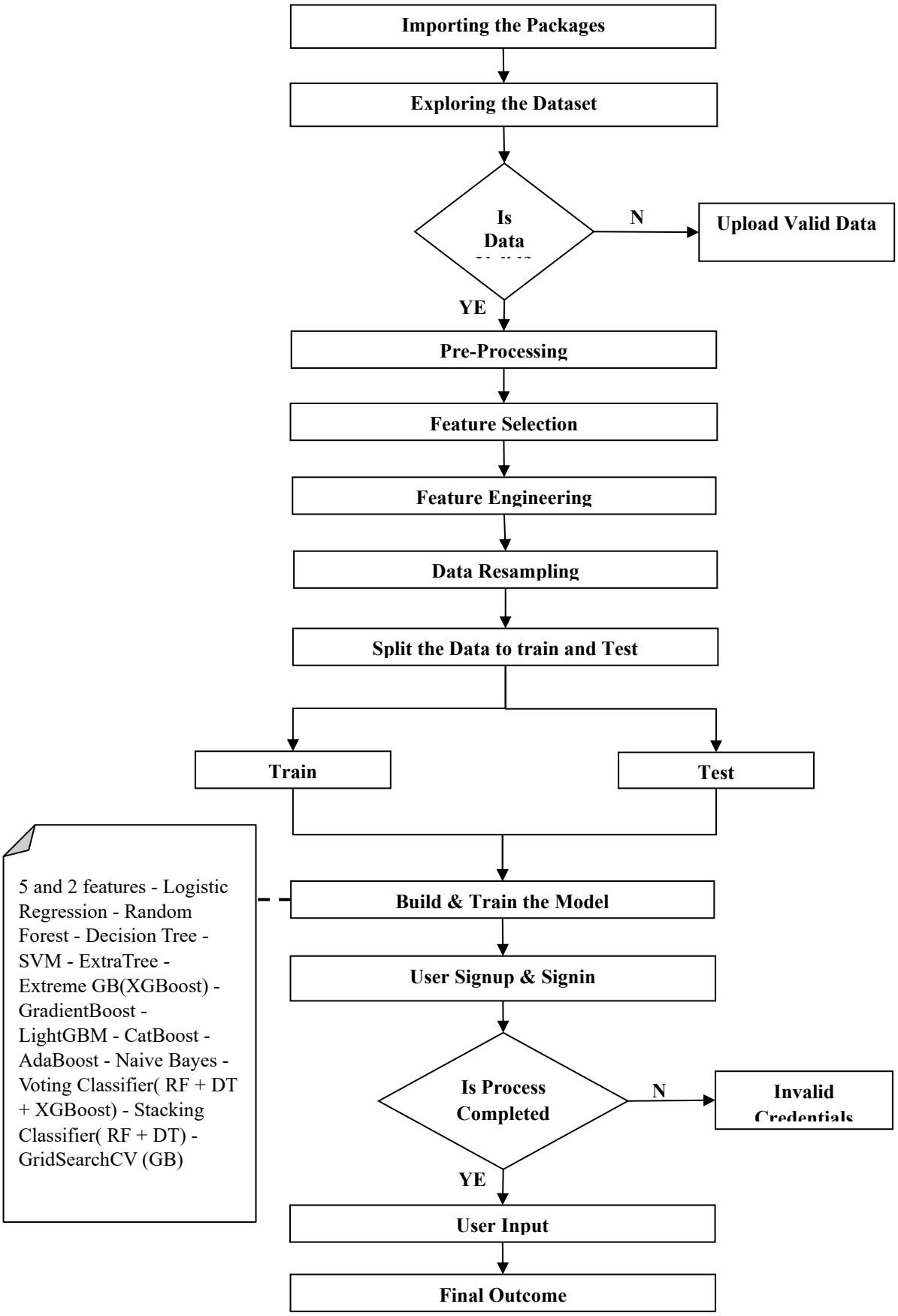
- Data Input → Preprocessing → Feature Selection → Model → Output

Main Processes

- Data collection
- Data preprocessing
- Model training
- Prediction generation

Data Stores

- Dataset storage
- User database
- Model storage


Importing the Packages
Exploring the Dataset
Is Data
N
Upload Valid Data
YE
Pre-Processing
Feature Selection
Feature Engineering
Data Resampling
Split the Data to train and Test
Train
Test
5 and 2 features - Logistic Regression - Random Forest - Decision Tree - SVM - ExtraTree - Extreme GB(XGBoost) - GradientBoost - LightGBM - CatBoost - AdaBoost - Naive Bayes - Voting Classifier(RF + DT + XGBoost) - Stacking Classifier(RF + DT) - GridSearchCV (GB)
Build & Train the Model
User Signup & Signin
Is Process Completed
N
Invalid
YE
User Input
Final Outcome

5.3. UML Diagrams

Unified Modeling Language (UML) is a standardized visual language used to represent the design and functionality of a system through diagrams. It helps in describing how different parts of the system interact with each other and how data flows between them. In the context of a sleep disorder prediction system, UML plays a crucial role in illustrating processes such as input handling, preprocessing, feature transformation, model training, and prediction generation in a simplified visual manner.

UML diagrams, such as use case diagrams, activity diagrams, and sequence diagrams, provide insights into how users interact with the system, how internal processes are triggered, and how results are delivered. For example, a use case diagram can show the interaction between a user and the prediction system, while an activity diagram highlights the steps involved in transforming raw input into meaningful diagnostic results.

By employing UML, the entire system can be understood in terms of processes and interactions rather than technical complexity. This makes it easier to communicate functionality, ensure completeness of processes, and maintain clarity about how user inputs translate into predictions. In sleep disorder prediction, UML ensures that the logical structure of the system is clearly defined and easy to interpret.

Goals of UML:

To provide a standardized way of visualizing the sleep disorder prediction system, ensuring clarity in how processes interact and data flows, making it easier to understand system functionality without delving into technical complexities.

To simplify communication of system behavior by representing user interactions, preprocessing, feature handling, and prediction generation, allowing a clear overview of how sleep disorder detection tasks are carried out step by step. To ensure transparency in representing processes, making it easier to verify that all essential steps for accurate sleep disorder prediction are included and no critical functionality is overlooked in the workflow.

To support better analysis of system requirements by visualizing dependencies between inputs, processes, and outputs, helping highlight potential improvements that strengthen reliability and efficiency in predicting sleep disorders using machine learning techniques.

To facilitate easier interpretation of complex processes through diagrams, ensuring that both technical and non-technical audiences can understand how inputs are transformed into predictions in the sleep disorder prediction system.

5.3.1. Use Case Diagrams

A use case diagram is a visual representation that shows the interaction between actors and use cases through associations. In this system, actors include patients and healthcare professionals who interact with the predictive model. Use cases represent functions like providing health details, preprocessing input, generating predictions, and displaying results. Associations highlight the relationship between user actions and system responses. By modeling these interactions, the diagram helps capture functional requirements clearly and illustrates how different users engage.

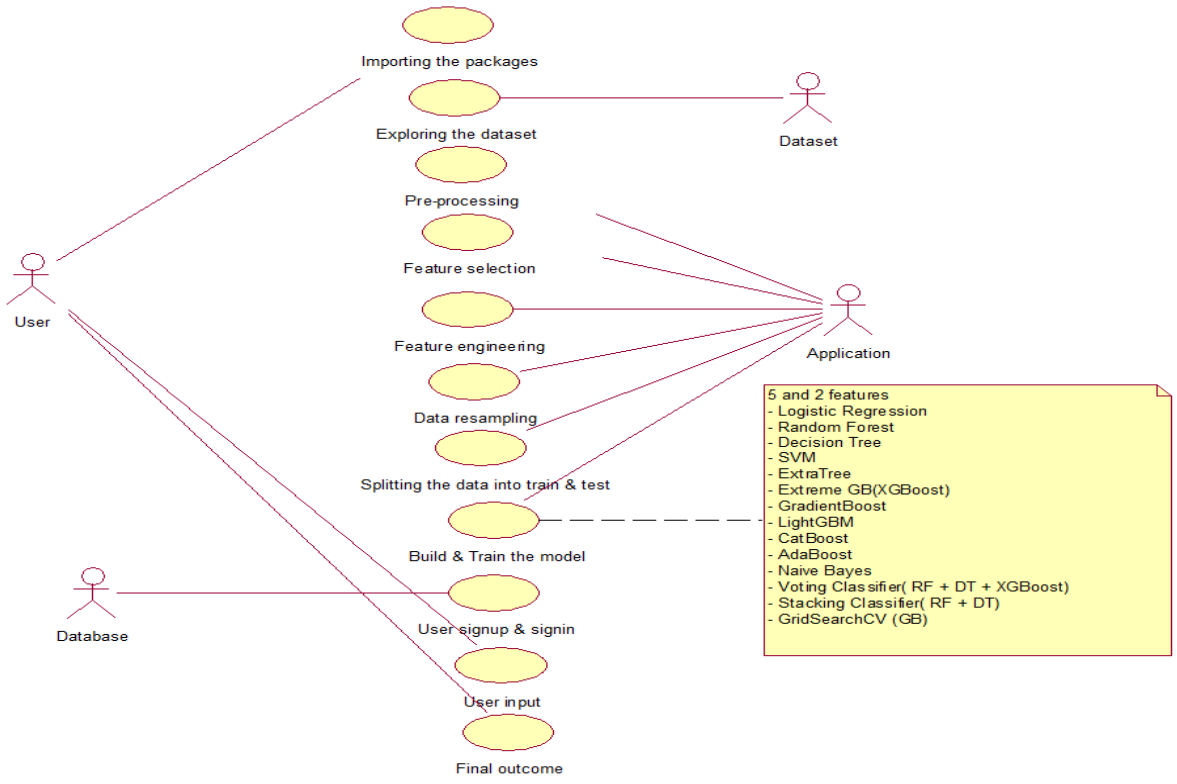


Fig. 5.3.1. Use Case Diagram

5.3.2. Class Diagram

A class diagram is used to represent the static structure of the system, showcasing classes, their attributes, operations, and relationships like aggregation. In this sleep disorder prediction system, classes may include User, Preprocessor, FeatureEngineer, Classifier, and Result. Attributes define properties such as user details or processed features, while operations specify actions like preprocess data, train model, or generate prediction. Aggregation can demonstrate how the Classifier aggregates features from Preprocessor and FeatureEngineer. This ensures clear representation of how different components structurally interact to support the prediction workflow.

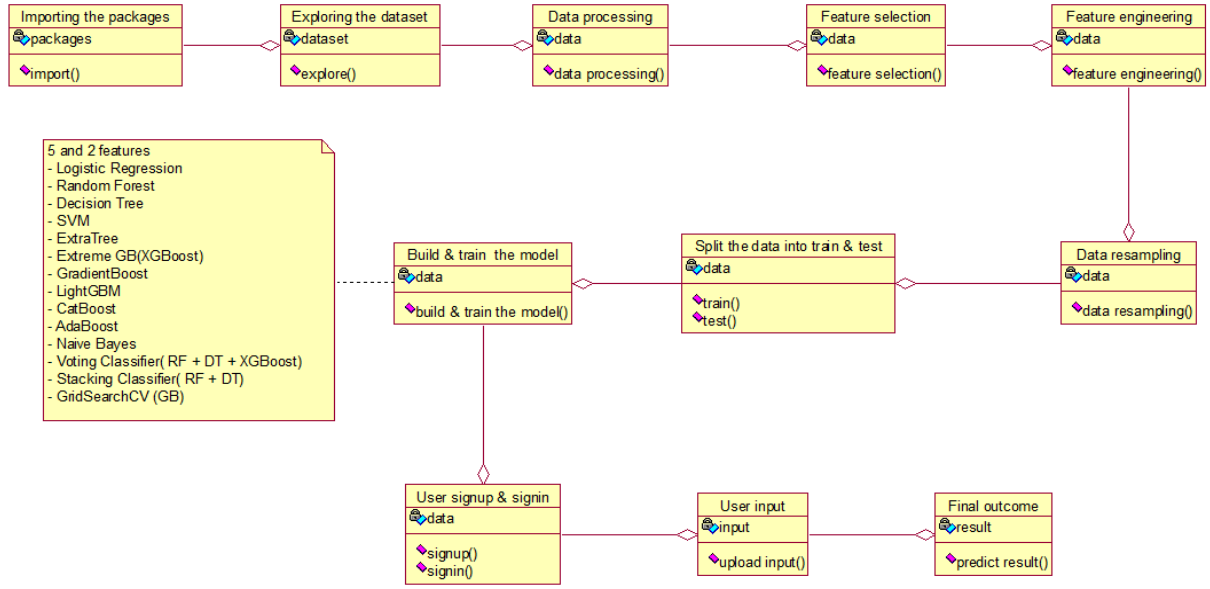


Fig.5.3.2. Class Diagram

5.3.3. Activity Diagram

An activity diagram illustrates workflows by focusing on actions, decisions, and control flow. For this system, actions include input submission, preprocessing, feature selection, model prediction, and result display. Decisions occur when checking data validity, handling missing values, or selecting features. Control flow connects these actions and decisions sequentially, demonstrating how one step leads to another until prediction is complete. The activity diagram provides a

dynamic view of the diagnostic process, ensuring all paths, including error handling and successful execution, are explicitly represented.

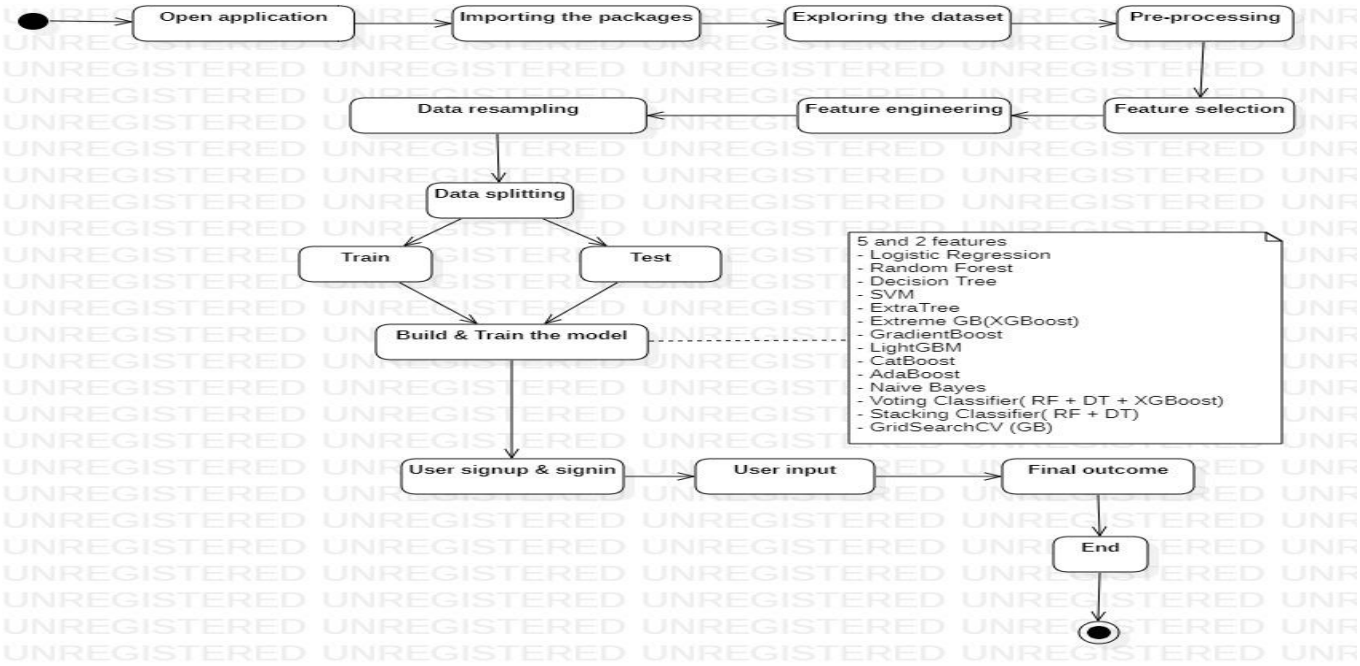


Fig.5.3.3. Activity Diagram

5.3.4. Sequence Diagram

A sequence diagram models dynamic interactions between objects over time, showing how object messages are exchanged. In this system, objects like User, InputHandler, Preprocessor, Classifier, and ResultViewer communicate sequentially. The User object sends a message with lifestyle data to the InputHandler, which passes cleaned data to the Preprocessor. Processed data is forwarded to the Classifier, which returns prediction results. The ResultViewer object then presents the outcome. This diagram clarifies the exact order of operations and message flows required to complete prediction tasks.

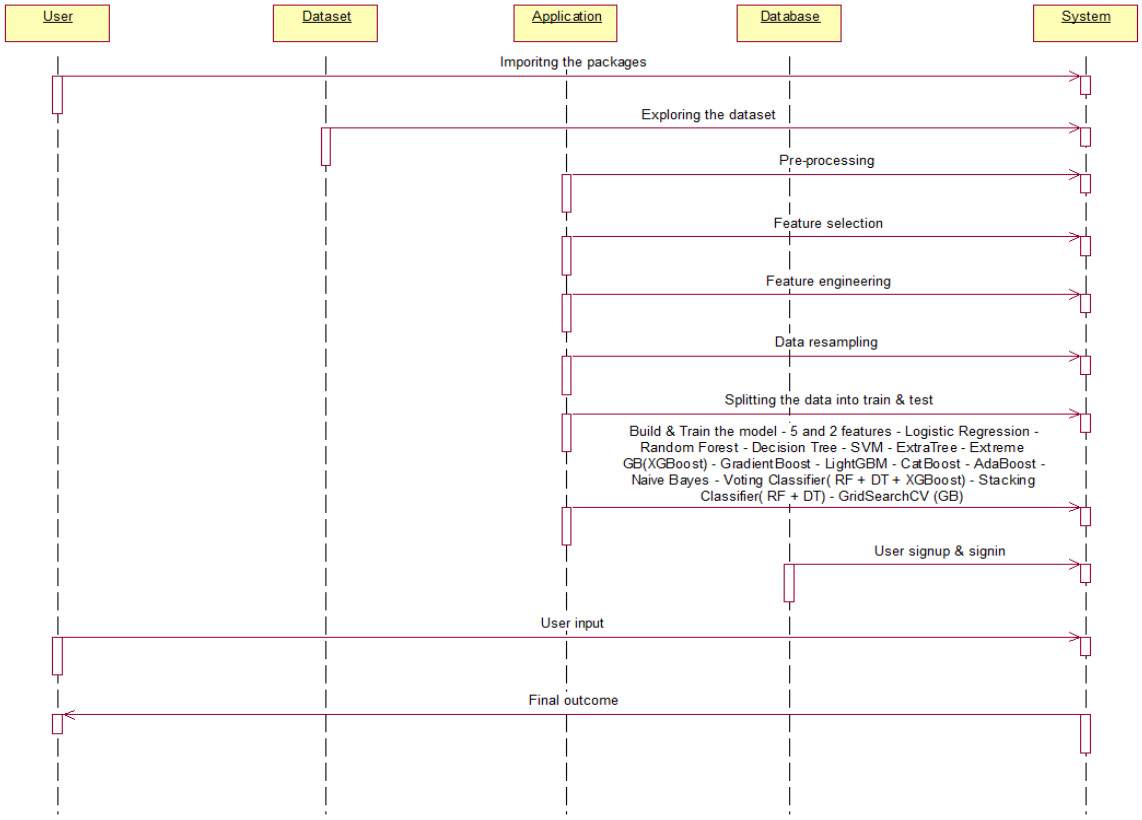


Fig.5.3.4. Sequence Diagram

5.3.5. Collaboration Diagram:

A collaboration diagram focuses on interactions between objects, emphasizing their structural relationships and link messages exchanged. Within this system, objects such as User, Preprocessor, FeatureEngineer, Classifier, and ResultViewer are connected through links. Messages describe how processed inputs move through preprocessing and classification stages to generate predictions. Unlike sequence diagrams, emphasis is placed on object organization and their communication network. This representation helps illustrate how components cooperate structurally to support diagnosis, ensuring the correct flow of health-related information between interconnected system objects.

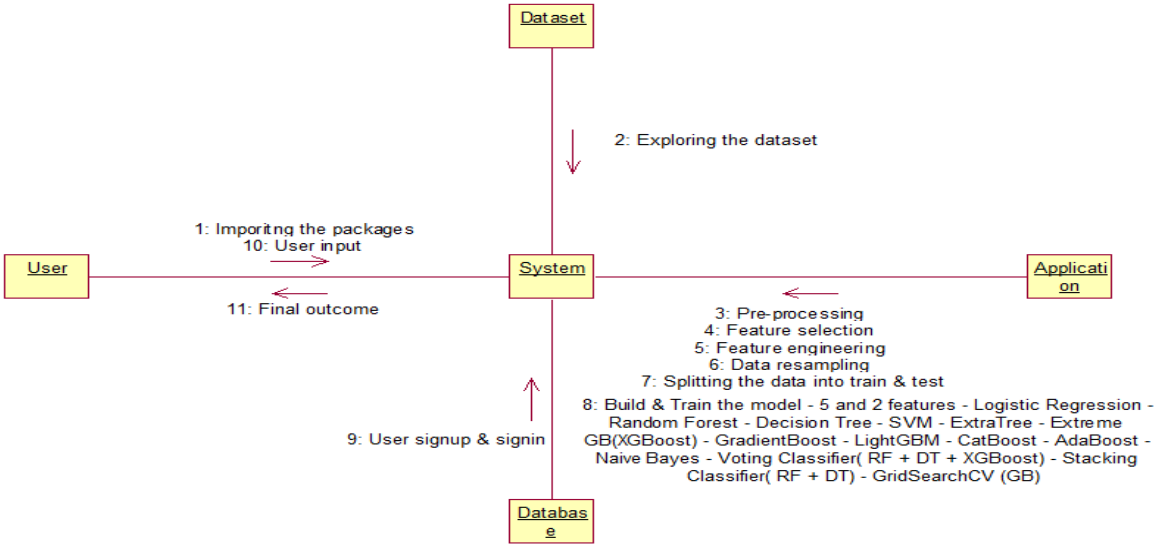


Fig.5.3.5. Collaboration Diagram

5.3.6 Component Diagram:

A component diagram depicts the modular design of a system by showing components, packages, and dependency relationships. For this system, components include Preprocessing Module, Feature Engineering Module, Model Training Module, and Prediction Interface. Each component can be grouped into packages for logical organization. Dependencies highlight how one module relies on outputs from another, such as Feature Engineering depending on Preprocessing. This ensures modularity and simplifies updates, as each component can be refined independently while still maintaining consistent system functionality for sleep disorder prediction.

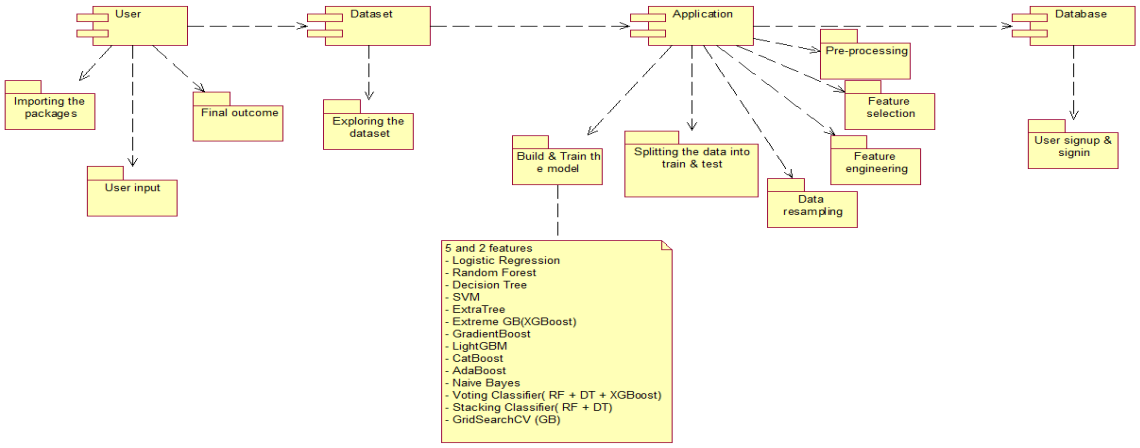


Fig. 5.3.6 Component Diagram

5.3.7 Deployment Diagram:

A deployment diagram illustrates the physical arrangement of system execution environments using nodes, connectors, notes, and anchored notes. In this case, nodes may include a client device for user input and a server node for processing and prediction tasks. Connectors represent communication between client and server. Notes provide clarifications about processing requirements or configurations, while anchored notes attach this information to specific items, such as the prediction module. This diagram ensures clarity about where and how different system elements are deployed for operational use.

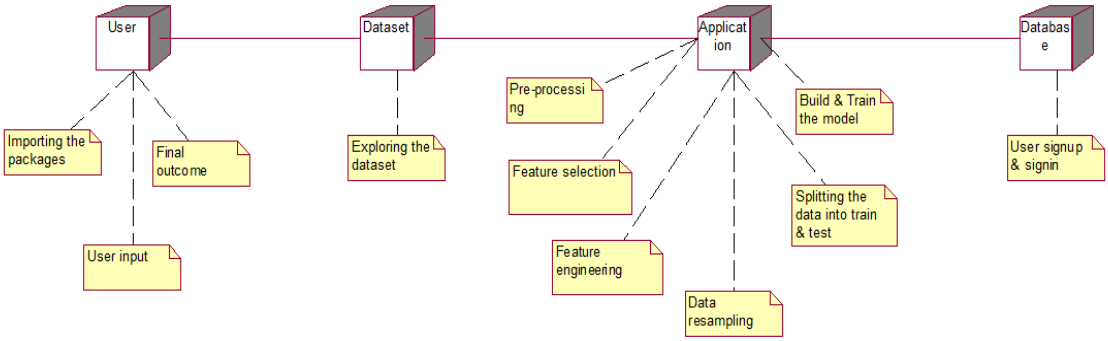


Fig. 5.3.7 Deployment Diagram:

5.4. Control Flow Diagram

A Control Flow Diagram (CFD) is a graphical representation that illustrates the sequence in which different operations and instructions are executed within a system. It focuses on the flow of control rather than the flow of data, showing how the system progresses from one stage to another and how decisions influence the execution path. In the Sleep Disorder Prediction System, the CFD plays a crucial role in defining the logical execution of processes, ensuring that each step is performed in a systematic and error-free manner.

Unlike Data Flow Diagrams, which emphasize the movement of data, Control Flow Diagrams highlight execution order, decision points, loops, and conditional branching, making them essential for understanding the internal working of the system.

Purpose of Control Flow Diagram

The primary purpose of the Control Flow Diagram is to:

- Represent the **step-by-step execution** of the system

- Identify **decision-making points** and conditional logic
- Ensure the **correct sequence of operations**
- Help in **debugging and error handling**
- Provide a clear understanding of system logic to developers

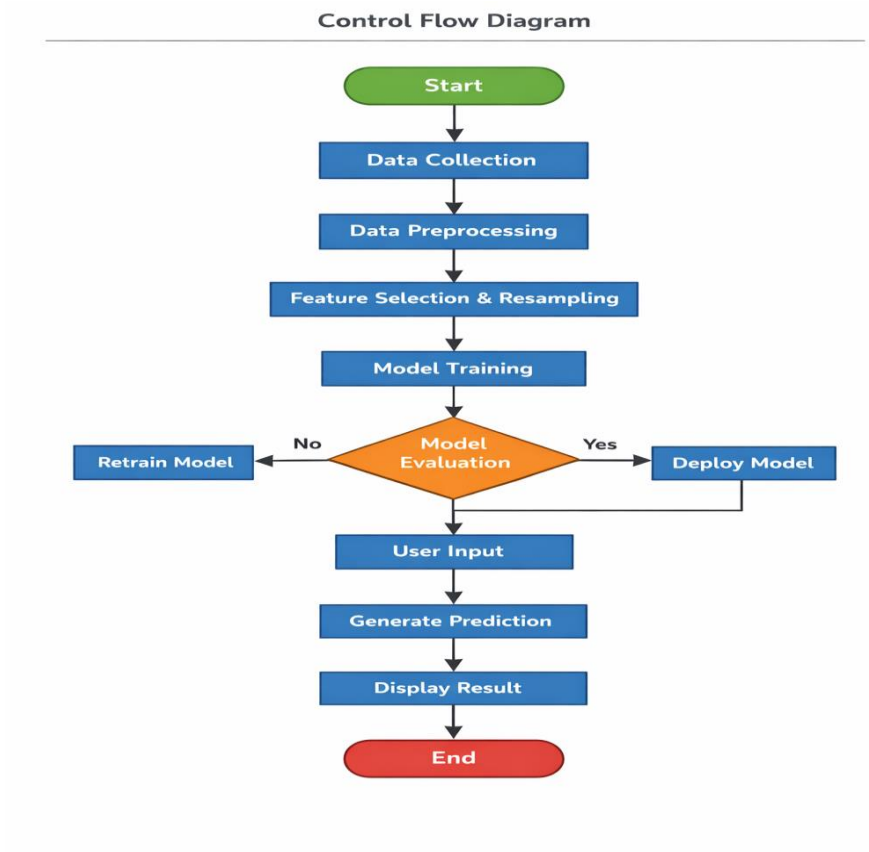


Fig.5.4. Control Flow Diagram

Advantages of Control Flow Diagram

- Provides a clear understanding of system logic
- Helps in identifying errors and inefficiencies
- Supports better program design and development
- Simplifies complex workflows
- Useful for testing and debugging

Importance of the Project

In the Sleep Disorder Prediction System, the CFD ensures that:

- Data processing steps are executed in the correct order
- Decision-making (like model evaluation) is handled properly
- The system responds correctly to user inputs
- Prediction is performed only after proper validation and train

5.5. Database Designs

Database design is the process of organizing and structuring data in a way that ensures efficient storage, retrieval, and management. In this system, a relational database is used to store user information, health data, and prediction results.

The database design plays a crucial role in storing, managing, and retrieving data efficiently for the sleep disorder detection system. The system handles patient details, sleep-related parameters, and prediction results. A structured relational database is used to ensure data consistency, accuracy, and scalability.

The database is designed to support machine learning models by storing both input features (such as lifestyle and health parameters) and output predictions (sleep disorder classification).

Objectives of Database Design

The main objectives are:

- To store patient and sleep-related data securely
- To maintain data consistency and avoid redundancy
- To enable efficient data retrieval for analysis and prediction
- To support scalability for future data expansion
- To ensure proper relationship between different data entities

Database Type

- **Relational Database (MySQL / SQLite)**

Advantages of Database Design

- Organized data storage
- Fast data retrieval
- Secure data handling

- Supports multiple users

ER Diagram (Explanation)

The Entity Relationship (ER) diagram consists of four main entities: Patient, Sleep_Data, Prediction, and User.

- Patient acts as the central entity
- Sleep_Data and Prediction are dependent entities
- Relationships are established using foreign keys

This structure ensures efficient data flow between input data and prediction output.

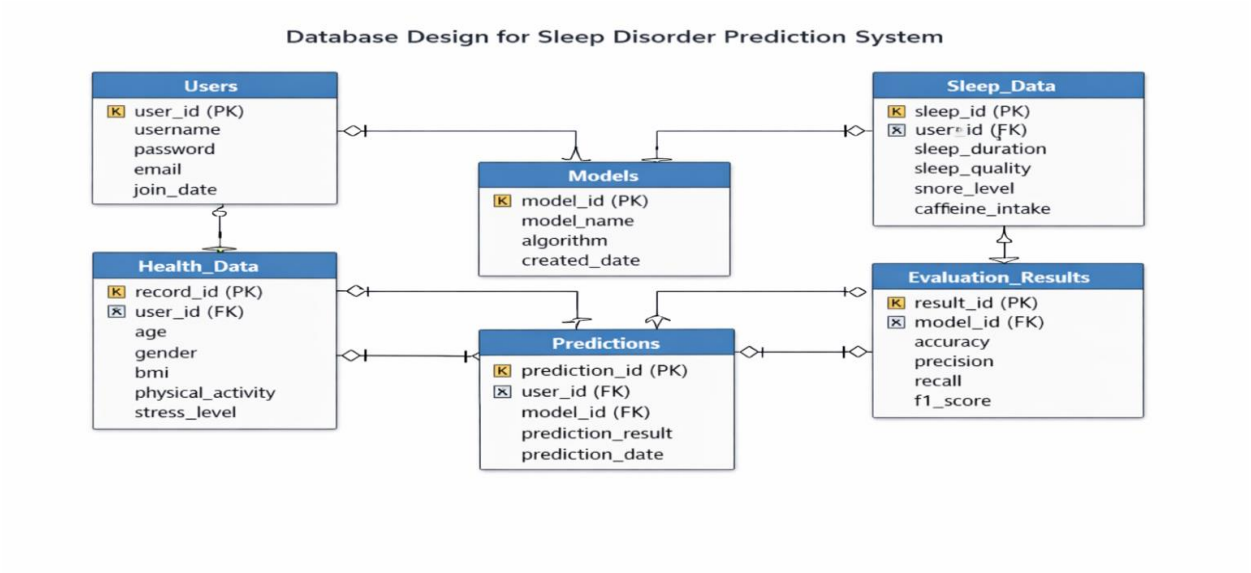


Fig.5.5. Database Diagram

CHAPTER-6: METHODOLOGY

The methodology of the Sleep Disorder Prediction System is designed to ensure accurate and efficient prediction of sleep disorders using a structured machine learning pipeline. The process begins with data collection and preprocessing, followed by feature selection, resampling, model training, evaluation, and deployment. Each stage is carefully designed to improve the performance, reliability, and interpretability of the system. Advanced techniques such as ANOVA for feature selection and SMOTEENN for handling class imbalance are incorporated to enhance model accuracy. The final system provides real-time predictions through a user-friendly web interface.

6.1. Technologies Overview

The project uses various tools and technologies to build an efficient and scalable system:

Python

Python is used as the main programming language due to its simplicity and extensive support for data science and machine learning. It is used for data preprocessing, model development, and deployment.

Scikit-learn

Scikit-learn is a powerful machine learning library used for implementing algorithms such as Logistic Regression, Decision Tree, Random Forest, and Support Vector Machine. It also provides tools for model evaluation and feature selection.

Flask

Flask is a lightweight web framework used to develop the web application interface. It allows users to input data and receive predictions easily through a browser.

MySQL

MySQL is used as a database management system to store user details, health data, and prediction results. It ensures efficient data storage and retrieval.

SMOTEENN

SMOTEENN is a hybrid resampling technique that combines SMOTE (oversampling) and Edited Nearest Neighbors (under sampling). It helps in balancing imbalanced datasets, improving model performance.

6.2. Methodology Overview

The design methodology of the proposed sleep disorder detection system focuses on creating a structured, efficient, and scalable architecture that integrates data processing, machine learning, and user interaction. The system is designed to transform raw input data into meaningful predictions through a well-defined pipeline. The design ensures modularity, flexibility, and ease of implementation while maintaining high performance and accuracy.

The methodology follows a layered approach, where each component performs a specific function and interacts seamlessly with other components. This structured design improves maintainability and allows future enhancements without affecting the overall system

Modular Design Approach

The system is divided into independent modules to improve efficiency and maintainability:

- Data Collection Module – Handles input data gathering
- Preprocessing Module – Cleans and prepares data
- Feature Selection Module – Identifies important features
- Model Training Module – Trains machine learning models
- Prediction Module – Generates results
- Database Module – Stores data and predictions

Each module operates independently but is connected through a defined workflow.

The system follows a step-by-step machine learning workflow:

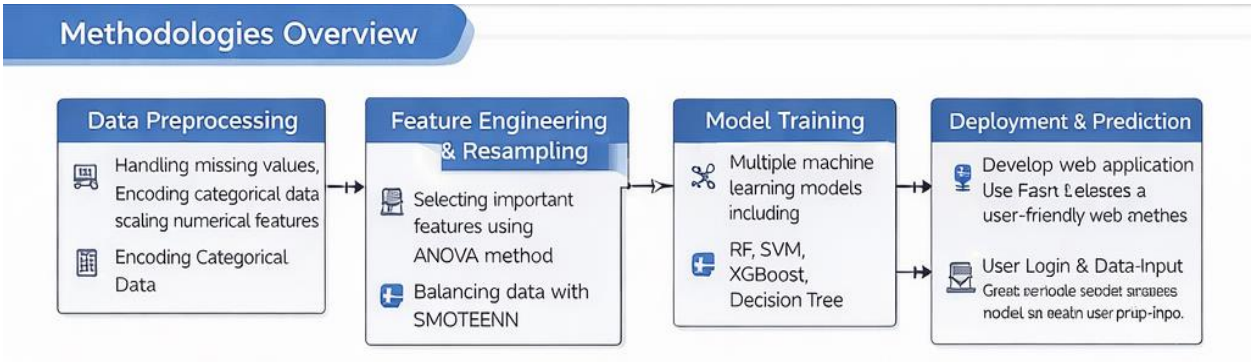


Fig.6.2. Methodology Overview

- Data Preprocessing

This step involves cleaning and preparing the dataset. Missing values are handled, categorical variables are encoded into numerical form, and numerical features are scaled to ensure consistency.

- Feature Engineering & Resampling

Important features are selected using the ANOVA method, which identifies the most relevant attributes affecting sleep disorders. SMOTEENN is applied to balance the dataset and improve prediction accuracy.

- Model Training

Multiple machine learning models are trained, including Random Forest, Decision Tree, Support Vector Machine, Logistic Regression, and XGBoost. This helps in comparing performance and selecting the best model.

- Deployment & Prediction

The trained model is deployed using Flask. Users can log in, enter their health details, and receive predictions about sleep disorders in real time.

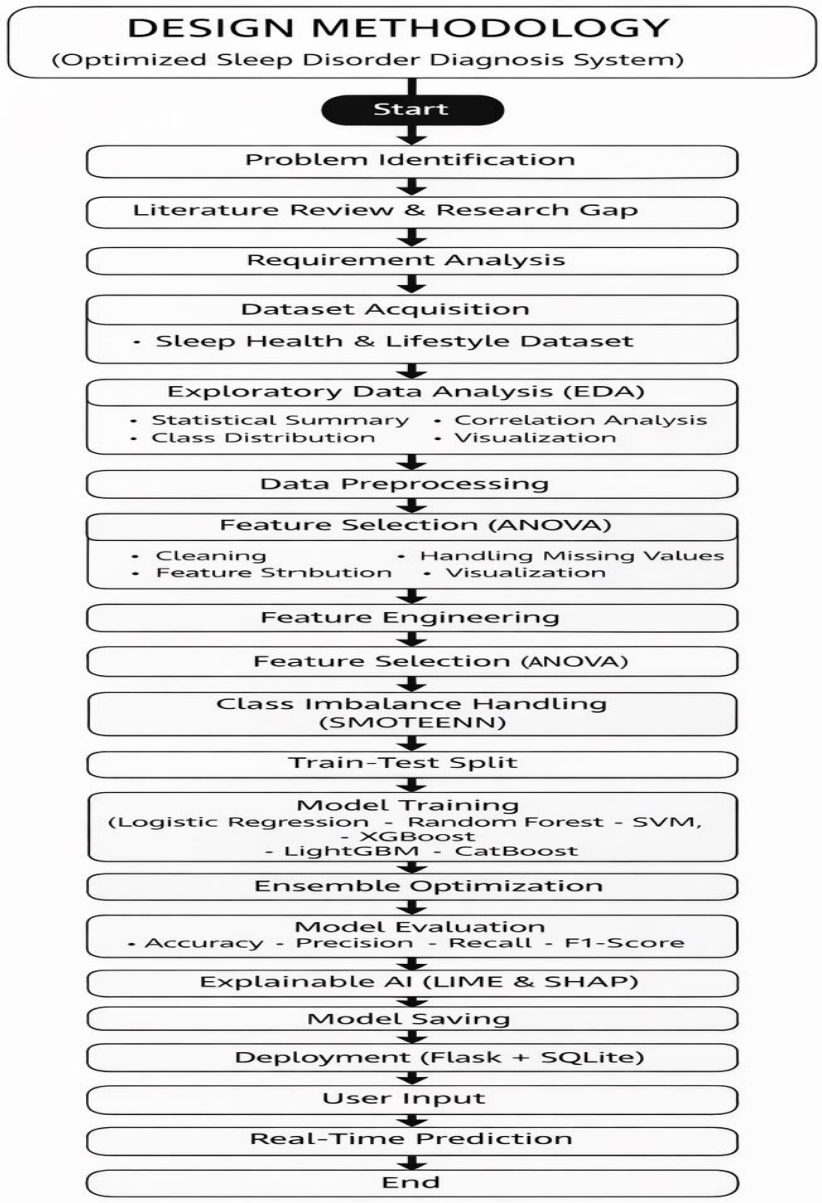


Fig.6.2. Design Methodology

6.3. Module Description

The proposed system is divided into several functional modules to ensure a structured and efficient workflow. Each module is designed to perform a specific task, contributing to the overall performance of the sleep disorder detection system. The modular approach improves system flexibility, maintainability, and scalability, allowing easy updates and enhancements in the future.

4.1 Problem Identification

The first step involves identifying the healthcare challenges associated with sleep disorders. Traditional diagnostic methods such as Polysomnography (PSG) are expensive, time-consuming, and require hospital infrastructure.

4.2 Dataset Acquisition

The system utilizes the *Sleep Health and Lifestyle Dataset*, which contains relevant physiological and lifestyle attributes. Important features such as Body Mass Index (BMI), stress levels, sleep duration, and physical activity are selected.

4.3 Exploratory Data Analysis (EDA)

Exploratory Data Analysis is performed to understand the structure and characteristics of the dataset. Statistical distributions are analyzed, correlations between variables are identified, and data imbalances are detected.

4.4 Data Preprocessing

Data preprocessing is carried out to improve data quality and ensure reliable model performance. This includes removal of duplicate records, handling missing values, encoding categorical variables, and normalizing numerical features.

These steps ensure that the dataset is clean, consistent, and suitable for machine learning algorithms.

4.5 Feature Selection (ANOVA)

Feature selection is performed using the Analysis of Variance (ANOVA) technique to identify the most significant predictors. Irrelevant or low-impact features are removed, while important variables such as stress level, sleep duration, and BMI are retained. This improves model efficiency and enhances prediction accuracy.

4.6 Feature Engineering

Feature engineering techniques are applied to enhance data representation. New features are derived, continuous variables are grouped, and health risk indicators are created. These transformations help in capturing hidden patterns and improving the detection of sleep disorders.

4.7 Class Imbalance Handling (SMOTEENN)

To address class imbalance in the dataset, the SMOTEENN technique is used. This method combines Synthetic Minority Over-sampling Technique (SMOTE) with Edited Nearest Neighbors (ENN) to generate synthetic samples and remove noisy data.This ensures that rare cases such as insomnia and sleep apnea are properly represented during model training.

4.8 Train-Test Split

The dataset is divided into training and testing sets to evaluate model generalization.

This prevents overfitting and ensures that the model performs effectively on unseen data.

4.9 Model Training

Multiple machine learning algorithms are trained using the processed dataset. The models learn patterns associated with sleep disorders, such as the relationship between high stress levels and insomnia or high BMI and sleep apnea. Hyperparameter tuning is performed to optimize model performance.

4.10 Ensemble Optimization

To improve prediction accuracy and stability, ensemble techniques such as voting or stacking are applied. These methods combine multiple models to produce more reliable and robust predictions.

4.11 Model Evaluation

The performance of the trained models is evaluated using standard metrics such as Accuracy, Precision, Recall, and F1-Score. Special emphasis is given to Recall, as it is critical in medical applications to minimize the chances of missing actual disorder cases.

4.12 Explainable AI (LIME and SHAP)

Explainable AI techniques such as LIME and SHAP are used to interpret model predictions. These methods identify feature importance and provide explanations for individual predictions. This helps in increasing transparency and allows medical professionals to understand the reasoning behind the predictions.

4.13 Model Saving

The trained model is serialized and stored for future use. This allows the system to reuse the trained model without retraining, enabling efficient deployment.

4.14 Deployment (Flask and SQLite)

The system is deployed using a Flask-based web application with SQLite database integration. The web interface allows users to input their health data and receive predictions.The database is used to store user inputs and prediction results for future reference.

4.15 User Input Module

The user interface collects real-time health data through a structured form. Input validation and formatting are performed before sending the data to the prediction engine.

4.16 Real-Time Prediction

The system processes user input, applies preprocessing steps, and generates predictions using the trained model.

CHAPTER-7: IMPLEMENTATION

The implementation phase focuses on converting the proposed design and methodology into a working system. It involves developing various modules, integrating machine learning algorithms, and creating a user-friendly interface for predicting sleep disorders. The system is implemented using efficient programming tools and technologies to ensure accuracy, reliability, and scalability. The implementation is carried out in a step-by-step manner, starting from data handling to final prediction output. Each component is carefully developed and tested to ensure smooth system functionality.

7.1. Modules

1. Importing the Packages: We begin by importing essential Python libraries including Pandas, NumPy, Scikit-learn, Matplotlib, and Seaborn for data handling, visualization, and preprocessing. Additionally, libraries such as XGBoost, LightGBM, and CatBoost are included for advanced machine learning model training, while Joblib is used to save and load trained artifacts efficiently.

2. Exploring the Dataset: The dataset is loaded into a Pandas DataFrame to inspect its structure, data types, and summary statistics. Exploration involves checking column names, identifying missing values, and analyzing distributions of features. Visualization techniques like histograms, correlation heatmaps, and bar plots help understand patterns in the data and initial feature relationships effectively.

3. Processing: Preprocessing ensures the dataset is clean and ready for modeling. Steps include handling missing values through imputation or dropping, encoding categorical variables using label encoding, and standardizing numerical features for uniform scaling. If data imbalance is present, appropriate sampling methods are applied to balance classes, ensuring better learning and fairness across predictions.

4. Feature Selection: Feature selection involves applying ANOVA statistical testing to identify the most significant features influencing the target variable. By filtering irrelevant or redundant attributes, the dimensionality of the dataset is reduced. This enhances computational efficiency, eliminates noise, and improves the accuracy and generalization capabilities of machine learning models trained on selected features.

5. Feature Engineering: To improve performance, seven new features were engineered using predictions from multiple classifiers: Random Forest, Gradient Boosting, Gaussian Naive Bayes, KNN, Decision Tree, Logistic Regression, and SVM. These engineered attributes capture hidden patterns, enrich dataset representation, and provide models with additional learning signals, strengthening predictive capability beyond original feature space.

6. Data Resampling: The SMOTEENN technique is applied for resampling, addressing dataset imbalance. SMOTE generates synthetic minority samples, while Edited Nearest Neighbors removes noisy or misclassified points. Combining both methods ensures a balanced dataset, reduces overfitting risks, and improves generalization, helping models learn effectively from both majority and minority classes without bias issues.

7. Split the Data into Train & Test: The processed dataset is divided into training and testing subsets. The training set is used to build models, while the testing set evaluates generalization. This separation prevents data leakage, ensures unbiased performance measurement, and helps validate the robustness of trained classifiers under unseen real-world prediction scenarios, maintaining fairness.

8. Model Training: Various classifiers are trained including Logistic Regression, Random Forest, Decision Tree, SVM, Extra Tree, XGBoost, Gradient Boosting, LightGBM, CatBoost, AdaBoost, Naive Bayes, Voting Classifier combining RF, DT, XGBoost, and Stacking Classifier combining RF and DT. Hyperparameter optimization is performed using GridSearchCV, especially for Gradient Boosting, ensuring maximum model performance and stability.

9. Evaluation: Models are evaluated using metrics including accuracy, precision, recall, F1-score, sensitivity, and specificity. Confusion matrices provide insights into true and false predictions. Training time is recorded to assess efficiency. Explainable AI techniques further enhance evaluation, ensuring interpretability of outcomes and establishing confidence in classifier decisions through transparent model understanding.

10. Flask Framework: The trained model is deployed using Flask. A lightweight web framework connects the backend prediction system to a user-friendly frontend. Flask handles preprocessing,

model loading, and prediction logic, ensuring smooth interaction between system components. The framework allows real-time predictions through a simple browser-based interface, enhancing accessibility and usability effectively.

11. User Signup & Signin: A user authentication module allows individuals to securely register and access the system. Signup collects details, while signin verifies credentials. This ensures personalized interactions, controlled access, and protects sensitive information. Proper session handling enables continuous interaction with the platform, enhancing trust and security during prediction-based operations and data handling.

12. User Input: Users provide input values for health and lifestyle features via a simple web form built in HTML. Flask backend preprocesses the inputs, scales numerical fields, encodes categorical values, and passes processed data to the model. Predictions are generated instantly and displayed, ensuring smooth, interactive, and transparent user experience consistently.

13. Final Outcome: The final system integrates preprocessing, resampling, feature selection, engineered features, model training, and Flask deployment into a seamless workflow. High-performance algorithms achieve reliable sleep disorder predictions. Users interact with a simple web interface, input their details, and instantly receive interpretable results, promoting practical application of machine learning advancements.

7.2. Algorithms

Logistic Regression: Logistic Regression is a statistical model used for binary or multiclass classification problems. It predicts the probability that an observation belongs to a specific class using the logistic function. This model is widely used due to its simplicity, interpretability, and efficiency for linearly separable data. In sleep disorder prediction, it helps establish baseline performance and provides interpretable relationships between features and outcomes. Its main purpose is to classify data points based on calculated probabilities and highlight the influence of predictors, making it effective for understanding health-related factors impacting classification outcomes.

Random Forest: Random Forest is an ensemble learning algorithm that builds multiple decision trees and combines their predictions to improve accuracy and reduce overfitting. Each tree is trained on random subsets of data and features, ensuring diversity. It is widely used in classification and regression tasks due to its robustness and scalability. In sleep disorder detection, Random Forest effectively handles complex, non-linear relationships between features. Its purpose is to enhance model performance, manage high-dimensional data, and provide feature importance measures, helping identify the most relevant factors contributing to prediction outcomes and improving the reliability of results.

Decision Tree: Decision Tree is a supervised learning algorithm that splits data into branches based on decision rules derived from feature values. It produces a tree-like structure where internal nodes represent conditions, branches indicate outcomes, and leaves provide final predictions. Widely used for classification tasks, it is intuitive and interpretable. For sleep disorder prediction, it helps map decision paths clearly. Its purpose is to classify observations by recursively partitioning the feature space, making it useful for understanding decision boundaries and explaining predictions in an easily interpretable format, despite sometimes being prone to overfitting on complex datasets.

Support Vector Machine (SVM): Support Vector Machine is a powerful supervised learning algorithm used for classification and regression. It works by finding the optimal hyperplane that maximizes the margin between classes in feature space. SVM is effective for high-dimensional datasets and can handle non-linear decision boundaries through kernel functions. In sleep disorder analysis, it helps capture complex feature interactions that simpler models may miss. Its purpose is to achieve strong generalization by focusing on critical support vectors, making it effective for distinguishing between closely related health categories and delivering robust predictions across varying feature distributions.

Extra Trees Classifier: Extra Trees Classifier, or Extremely Randomized Trees, is an ensemble learning method similar to Random Forest but introduces greater randomness in splitting nodes. Instead of selecting the best split, it chooses random thresholds for each feature, improving generalization and reducing variance. This approach speeds up training while maintaining

accuracy. In sleep disorder prediction, it effectively handles noisy or complex datasets. Its purpose is to reduce overfitting risks, provide fast training, and enhance predictive reliability, especially in cases where the dataset includes multiple interacting features that may otherwise complicate simpler tree-based methods.

Extreme Gradient Boosting (XGBoost): Extreme Gradient Boosting, or XGBoost, is an advanced implementation of gradient boosting optimized for performance and efficiency. It builds sequential decision trees where each tree corrects the errors of the previous ones. XGBoost supports regularization to control overfitting and is highly scalable for large datasets. In predicting sleep disorders, it captures complex feature relationships with high accuracy. Its purpose is to enhance predictive power by combining weak learners into a strong ensemble, making it suitable for solving classification problems requiring high performance, precision, and generalization across diverse and imbalanced datasets.

Gradient Boosting: Gradient Boosting is an ensemble algorithm that builds models sequentially, where each model attempts to correct errors from its predecessors. It optimizes a loss function using gradient descent, producing a strong predictive model from multiple weak learners, usually decision trees. It is widely used for classification due to its flexibility and high accuracy. In sleep disorder classification, it captures subtle patterns between health and lifestyle features. Its purpose is to reduce prediction errors, enhance model accuracy, and provide reliable classification, especially in complex datasets with non-linear relationships among multiple interacting features.

LightGBM: LightGBM, or Light Gradient Boosting Machine, is a fast, efficient implementation of gradient boosting designed for large datasets. It uses a leaf-wise tree growth strategy with depth limitations, which improves accuracy while reducing computation. LightGBM supports parallel training, making it highly scalable and resource-efficient. In sleep disorder prediction, it is useful for handling high-dimensional features with speed and precision. Its purpose is to provide highly accurate predictions with optimized memory usage and reduced training time, making it suitable for real-time applications where fast and reliable classification is required.

CatBoost: CatBoost is a gradient boosting algorithm developed to handle categorical features efficiently without extensive preprocessing. It builds decision trees sequentially, reducing errors iteratively. Unlike other boosting techniques, it avoids overfitting with ordered boosting and supports high accuracy on diverse datasets. In sleep disorder prediction, CatBoost reduces the need for manual feature encoding, enhancing workflow efficiency. Its purpose is to simplify model building while providing robust, accurate predictions. It is particularly valuable when categorical data strongly influences outcomes, making it suitable for health and lifestyle-based classification tasks.

AdaBoost: AdaBoost, or Adaptive Boosting, is an ensemble learning algorithm that combines multiple weak classifiers, typically decision trees, into a strong model. It assigns higher weights to misclassified instances in each iteration, forcing subsequent learners to focus on difficult cases. AdaBoost is widely used for improving accuracy on binary classification tasks. In sleep disorder analysis, it enhances predictive performance by concentrating on complex data points. Its purpose is to increase model robustness, reduce bias, and improve overall generalization by adaptively adjusting to errors during training, resulting in stronger classification performance.

Naive Bayes: Naive Bayes is a probabilistic classifier based on Bayes' theorem, assuming independence between features. Despite its simplicity, it performs effectively on classification tasks, especially with categorical data. It calculates the probability of class membership given observed feature values. In sleep disorder prediction, it is efficient for quick, baseline classification. Its purpose is to provide a lightweight, interpretable model that works well with smaller datasets and serves as a benchmark. Though it assumes independence, it often performs surprisingly well in practical applications, offering insights into the relationships between features and class labels.

Voting Classifier: Voting Classifier is an ensemble technique that combines multiple models and aggregates their predictions to make a final decision. It can use hard voting, selecting the majority class, or soft voting, averaging predicted probabilities. In this system, Random Forest, Decision Tree, and XGBoost are combined for robust results.

Stacking Classifier: Stacking Classifier is an ensemble method that combines multiple base models and uses their predictions as inputs for a higher-level meta-model. This layered approach captures diverse patterns that individual models may miss. In this work, Random Forest and Decision Tree serve as base models, with a meta-classifier integrating results.

GridSearchCV: GridSearchCV is a hyperparameter optimization technique that exhaustively searches over a specified parameter grid for machine learning models. It trains and evaluates models on different parameter combinations using cross-validation, ensuring the selection of the most effective settings.

7.3. Source Code

app.py

```
import warnings
warnings.filterwarnings('ignore')
from flask import Flask, request, jsonify, render_template, redirect, url_for
import joblib
import sqlite3
import numpy as np
app = Flask(__name__)
@app.route('/')
def index():
    return render_template('index.html')
@app.route('/home1')
def home1():
    return render_template('home1.html')
@app.route('/home2')
def home2():
    return render_template('home2.html')
@app.route('/about1')
def about1():
```

```
    return render_template('about1.html')
@app.route('/about2')
def about2():
    return render_template('about2.html')
@app.route('/logon')
def logon():
    return render_template('signup.html')
@app.route('/login')
def login():
    return render_template('signin.html')
@app.route("/signup", methods=["GET", "POST"])
def signup():
    if request.method == "GET":
        return render_template("signup.html")
    else:
        username = request.form.get('user','')
        name = request.form.get('name','')
        email = request.form.get('email','')
        number = request.form.get('mobile','')
        password = request.form.get('password','')
        con = sqlite3.connect('signup.db')
        cur = con.cursor()
        cur.execute("SELECT 1 FROM info WHERE user = ?", (username,))
        if cur.fetchone():
            con.close()
            return render_template("signup.html", message="Username already exists. Please
choose another.")
        cur.execute("insert into `info` (`user`,`name`, `email`,`mobile`,`password`) VALUES (?, ?, ?, ?,
?)",(username,name,email,number,password))
```

```
        con.commit()
        con.close()
        return redirect(url_for('login'))
@app.route("/signin", methods=["GET", "POST"])
def signin():
    if request.method == "GET":
        return render_template("signin.html")
    else:
        mail1 = request.form.get('user','')
        password1 = request.form.get('password','')
        con = sqlite3.connect('signup.db')
        cur = con.cursor()
        cur.execute("select `user`, `password` from info where `user` = ? AND `password` =
?",(mail1,password1,))
        data = cur.fetchone()
        if data == None:
            return render_template("signin.html", message="Invalid username or password.")
        elif mail1 == 'admin' and password1 == 'admin':
            return render_template("home1.html")
        elif mail1 == str(data[0]) and password1 == str(data[1]):
            return render_template("home1.html")
        else:
            return render_template("signin.html", message="Invalid username or password.")
@app.route('/predict1', methods=['POST'])
def predict1():
    int_features = [float(x) for x in request.form.values()]
    final4 = [np.array(int_features)]
    model = joblib.load('model/eng_5_vc1.sav')
    prediction = int(model.predict(final4))
```

```
    if prediction == 0:  # Insomnia
        context = {
            "disorder_name": "Insomnia",
            "title_message": "About Insomnia",
            "disorder_info": "Insomnia is a common sleep disorder that can make it hard to fall asleep, hard to stay asleep, or cause you to wake up too early and not be able to get back to sleep.",
            "recommendations": [
                "Stick to a regular sleep schedule.",
                "Avoid caffeine and nicotine, especially before bed.",
                "Create a restful environment.",
                "Limit daytime naps."
            ],
            "border_color": "#dc3545",
            "header_bg_class": "bg-danger",
            "icon_class": "bx-alarm-exclamation"
        }
        return render_template('result1.html', **context)
    elif prediction == 1:  # No disorder
        context = {
            "disorder_name": "No Disorder Detected",
            "title_message": "Great News!",
            "disorder_info": "Based on the provided data, our AI model did not detect any signs of common sleep disorders. Your sleep patterns appear to be healthy.",
            "recommendations": [
                "Maintain a consistent sleep schedule, even on weekends.",
                "Create a relaxing bedtime routine to help you unwind.",
                "Ensure your bedroom is dark, quiet, and cool.",
                "Limit exposure to screens (phones, TVs) before bed."
            ],
```

```
        "border_color": "#28a745",
        "header_bg_class": "bg-success",
        "icon_class": "bx-check-circle"
    }
    return render_template('result1.html', **context)
elif prediction == 2:  # Sleep Apnea
    context = {
        "disorder_name": "Sleep Apnea",
        "title_message": "About Sleep Apnea",
        "disorder_info": "Sleep apnea is a potentially serious sleep disorder in which breathing repeatedly stops and starts. If you snore loudly and feel tired even after a full night's sleep, you might have sleep apnea.",
        "recommendations": [
            "Maintain a healthy weight.",
            "Avoid alcohol and sleeping pills.",
            "Sleep on your side or stomach rather than your back.",
            "Use a continuous positive airway pressure (CPAP) device if prescribed."
        ],
        "border_color": "#dc3545",
        "header_bg_class": "bg-danger",
        "icon_class": "bx-alarm-exclamation"
    }
    return render_template('result1.html', **context)
@app.route('/predict2', methods=['POST'])
def predict2():
    int_features = [float(x) for x in request.form.values()]
    final4 = [np.array(int_features)]
    model = joblib.load('model/eng_2_vc1.sav')
    prediction = int(model.predict(final4))
```

```
    if prediction == 0:  # Insomnia
        context = {
            "disorder_name": "Insomnia",
            "title_message": "About Insomnia",
            "disorder_info": "Insomnia is a common sleep disorder that can make it hard to fall asleep, hard to stay asleep, or cause you to wake up too early and not be able to get back to sleep.",
            "recommendations": [
                "Stick to a regular sleep schedule.",
                "Avoid caffeine and nicotine, especially before bed.",
                "Create a restful environment.",
                "Limit daytime naps."
            ],
            "border_color": "#dc3545",
            "header_bg_class": "bg-danger",
            "icon_class": "bx-alarm-exclamation"
        }
        return render_template('result2.html', **context)
    elif prediction == 1:  # No disorder
        context = {
            "disorder_name": "No Disorder Detected",
            "title_message": "Great News!",
            "disorder_info": "Based on the provided data, our AI model did not detect any signs of common sleep disorders. Your sleep patterns appear to be healthy.",
            "recommendations": [
                "Maintain a consistent sleep schedule, even on weekends.",
                "Create a relaxing bedtime routine to help you unwind.",
                "Ensure your bedroom is dark, quiet, and cool.",
                "Limit exposure to screens (phones, TVs) before bed."
            ],
```

```
        "border_color": "#28a745",
        "header_bg_class": "bg-success",
        "icon_class": "bx-check-circle"
    }
    return render_template('result2.html', **context)
elif prediction == 2:  # Sleep Apnea
    context = {
        "disorder_name": "Sleep Apnea",
        "title_message": "About Sleep Apnea",
        "disorder_info": "Sleep apnea is a potentially serious sleep disorder in which breathing repeatedly stops and starts. If you snore loudly and feel tired even after a full night's sleep, you might have sleep apnea.",
        "recommendations": [
            "Maintain a healthy weight.",
            "Avoid alcohol and sleeping pills.",
            "Sleep on your side or stomach rather than your back.",
            "Use a continuous positive airway pressure (CPAP) device if prescribed."
        ],
        "border_color": "#dc3545",
        "header_bg_class": "bg-danger",
        "icon_class": "bx-alarm-exclamation"
    }
    return render_template('result2.html', **context)
@app.errorhandler(404)
def not_found(e):
    return render_template('notfound.html'), 404
if __name__ == "__main__":
    app.run(debug=True)
```

code:

```
import numpy as np # linear algebra

import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/" directory

# For example, running this (by clicking run or pressing Shift+Enter) will list all files under the input directory

import os

for dirname, _, filenames in os.walk('/kaggle/input'):

    for filename in filenames:

        print(os.path.join(dirname, filename))
import matplotlib.pyplot as plt

import seaborn as sns

import plotly.express as px

import warnings

warnings.filterwarnings('ignore')
from sklearn.preprocessing import OneHotEncoder,LabelEncoder ,RobustScaler,StandardScaler

from sklearn.compose import ColumnTransformer

from sklearn.pipeline import Pipeline

fromsklearn.model_selectionimporttrain_test_split,StratifiedShuffleSplit,StratifiedKFold,cross_val_score

from imblearn.over_sampling import SMOTE

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, classification_report, confusion_matrix

from sklearn.linear_model import LogisticRegression

import xgboost as xgb
```

```
df = pd.read_csv('/kaggle/input/sleep-health-and-lifestyle-dataset/Sleep_health_and_lifestyle_dataset.csv')

df.head(5)

df.shape

df.dtypes

df.describe()

columns = [column for column in df.columns if column!='Person ID']

for column in columns:

    unique_values = df[column].unique()

    print(f"Unique values in '{column}': {unique_values}")

# Replace NaN in 'Sleep Disorder' with 'No Disorder'

df['Sleep Disorder'].fillna('No Disorder', inplace=True)

# Value counts of 'Sleep Disorder'

sleep_disorder_counts = df['Sleep Disorder'].value_counts()

print("Value counts of 'Sleep Disorder':")

print(sleep_disorder_counts)

df['BMI Category']=df['BMI Category'].replace({'Normal':'Normal Weight'})

df['BMI Category'].value_counts()

# bmi_counts = df['BMI Category'].value_counts()

fig = px.histogram(data_frame=df, x = 'BMI Category', color = 'Sleep Disorder', title='Bar Chart of BMI Category Counts')

fig.show()

fig = px.histogram(data_frame=df, x = 'Quality of Sleep', color = 'Sleep Disorder', title='Bar Chart of BMI Category Counts')

fig.show()

fig = px.histogram(data_frame=df, x = 'Occupation', color = 'Sleep Disorder', title='Bar Chart of BMI Category Counts')
```

```
fig.show()

color_palette = {'Male': 'lightblue', 'Female': 'lightcoral'}

plt.figure(figsize=(10, 6))

sns.violinplot(x='Gender', y='Quality of Sleep', data=df, palette=color_palette)

plt.title('Distribution of Quality of Sleep by Gender', fontsize=16)

plt.xlabel('Gender', fontsize=12)

plt.ylabel('Quality of Sleep', fontsize=12)

plt.show()

fig = px.bar(df,

        x='Stress Level',

        y='Quality of Sleep',

        color='Stress Level',

        title='Relationship between Stress Level and Quality of Sleep'

       )

fig.show()

df = pd.concat([df, df['Blood Pressure'].str.split('/', expand=True)], axis=1).drop('Blood Pressure',
axis=1)

df = df.rename(columns={0: 'Systolic', 1: 'Diastolic'})

df['Systolic'] = df['Systolic'].astype(float)

df['Diastolic'] = df['Diastolic'].astype(float)

numeric_features = ['Age', 'Sleep Duration',

            'Physical Activity Level',

            'Heart Rate', 'Daily Steps', 'Systolic', 'Diastolic']

# Calculate the correlation matrix

corr_matrix = df[numeric_features].corr()
```

```
# Set up the matplotlib figure

plt.figure(figsize=(12, 8))

# Draw the heatmap without the mask

sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f", vmin=-1, vmax=1, square=True)

# Customize plot labels and title

plt.title('Correlation Heatmap of data')

plt.xticks(rotation=45)

plt.yticks(rotation=0)

# Show plot

plt.show()df.drop(columns=['Person ID'],inplace = True)

label_encoder = LabelEncoder()

df['Sleep Disorder'] = label_encoder.fit_transform(df['Sleep Disorder'])

print(label_encoder.classes_)

numeric_features = ['Age', 'Sleep Duration',

                    'Heart Rate', 'Daily Steps', 'Systolic', 'Diastolic']

categorical_features = ['Occupation','Quality of Sleep','Gender',

                    'Physical Activity Level', 'Stress Level', 'BMI Category']

preprocessor = ColumnTransformer(

    transformers=[

        ('num', RobustScaler(), numeric_features),

        ('cat', OneHotEncoder(drop='first',sparse_output =False, handle_unknown='ignore'), categorical_features)

    ])

X = df.drop(columns=['Sleep Disorder'])
```

```
y = df['Sleep Disorder']

X_preprocessed = preprocessor.fit_transform(X)

# Initialize SMOTE

smote = SMOTE(random_state=42)

# Perform SMOTE oversampling

X_smote, y_smote = smote.fit_resample(X_preprocessed, y)

X_smote.shape

# Splitting the data into training and testing sets (e.g., 75% training, 25% testing)

X_train, X_test, y_train, y_test = train_test_split(X_smote, y_smote, test_size=0.25,
random_state=42)

# Initialize Logistic Regression classifier

model_lr = LogisticRegression()

# Fit the model on the training data

model_lr.fit(X_train, y_train)

# Predict on the test data

y_pred_lr = model_lr.predict(X_test)


# Calculate evaluation metrics

accuracy_lr = accuracy_score(y_test, y_pred_lr)

precision_lr = precision_score(y_test, y_pred_lr, average='weighted')

recall_lr = recall_score(y_test, y_pred_lr, average='weighted')

f1_lr = f1_score(y_test, y_pred_lr, average='weighted')

# Print metrics

print(f'Accuracy: {accuracy_lr}')

print(f'Precision: {precision_lr}')
```

```
print(f'Recall: {recall_lr}')
print(f'F1-score: {f1_lr}')
# Generate classification report
print(classification_report(y_test, y_pred_lr))
# Generate confusion matrix
cm_lr = confusion_matrix(y_test, y_pred_lr)
print('Confusion Matrix:')
print(cm_lr)
# Plot confusion matrix using seaborn
plt.figure(figsize=(8, 6))
sns.heatmap(cm_lr, annot=True, fmt='d', cmap='Blues', cbar=False)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()
# Initialize XGBoost classifier (assuming classification task)
model_xgb = xgb.XGBClassifier()
# Fit the model on the training data
model_xgb.fit(X_train, y_train)
# Predict on the test data
y_pred = model_xgb.predict(X_test)
# Calculate evaluation metrics
accuracy_xgb = accuracy_score(y_test, y_pred)
precision_xgb = precision_score(y_test, y_pred, average='weighted')
```

```
recall_xgb = recall_score(y_test, y_pred, average='weighted')

f1_xgb = f1_score(y_test, y_pred, average='weighted')

# Print metrics

print(f'Accuracy: {accuracy_xgb}')

print(f'Precision: {precision_xgb}')

print(f'Recall: {recall_xgb}')

print(f'F1-score: {f1_xgb}')

# Generate classification report

print(classification_report(y_test, y_pred))

# Generate confusion matrix

cm = confusion_matrix(y_test, y_pred)

print('Confusion Matrix:')

print(cm)

# Plot confusion matrix using seaborn

plt.figure(figsize=(8, 6))

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False)

plt.xlabel('Predicted')

plt.ylabel('Actual')

plt.title('Confusion Matrix')

plt.show()

xgb_model = xgb.XGBClassifier(random_state=42, use_label_encoder=False,
eval_metric='mlogloss')

# Define cross-validation strategy

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Perform cross-validation
```

```
scores = cross_val_score(xgb_model, X_smote, y_smote, cv=cv, scoring='accuracy')

scores

from sklearn.ensemble import GradientBoostingClassifier

# Initialize GBM classifier

gbm_clf = GradientBoostingClassifier(random_state=42)

# Train the model

gbm_clf.fit(X_train, y_train)

# Predictions

y_pred = gbm_clf.predict(X_test)

# Calculate metrics

accuracy_gbm = accuracy_score(y_test, y_pred)

precision_gbm = precision_score(y_test, y_pred, average='weighted')

recall_gbm = recall_score(y_test, y_pred, average='weighted')

f1_gbm = f1_score(y_test, y_pred, average='weighted')

# Classification report

print("Classification Report:")

print(classification_report(y_test, y_pred))

# Confusion matrix

cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(8, 6))

sns.heatmap(cm, annot=True, cmap='Blues', fmt='d', xticklabels=['No Disorder', 'Sleep Apnea',
'Insomnia'], yticklabels=['No Disorder', 'Sleep Apnea', 'Insomnia'])

plt.xlabel('Predicted')

plt.ylabel('Actual')

plt.title('Confusion Matrix - Gradient Boosting Machine')
```

```
plt.show()
# Display metrics
print(f"Accuracy: {accuracy_gbm:.4f}")
print(f"Precision: {precision_gbm:.4f}")
print(f"Recall: {recall_gbm:.4f}")
print(f"F1 Score: {f1_gbm:.4f}")
from sklearn.neighbors import KNeighborsClassifier
# Initialize KNN classifier (example using k=5)
knn_clf = KNeighborsClassifier(n_neighbors=5)
# Train the model
knn_clf.fit(X_train, y_train)
# Predictions
y_pred = knn_clf.predict(X_test)
# Calculate metrics
accuracy_knn = accuracy_score(y_test, y_pred)
precision_knn = precision_score(y_test, y_pred, average='weighted')
recall_knn = recall_score(y_test, y_pred, average='weighted')
f1_knn = f1_score(y_test, y_pred, average='weighted')
# Classification report
print("Classification Report:")
print(classification_report(y_test, y_pred))
# Confusion matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(8, 6))
```

```
sns.heatmap(cm, annot=True, cmap='Greens', fmt='d', xticklabels=['No Disorder', 'Sleep Apnea',
'Insomnia'], yticklabels=['No Disorder', 'Sleep Apnea', 'Insomnia'])

plt.xlabel('Predicted')

plt.ylabel('Actual')

plt.title('Confusion Matrix - K-Nearest Neighbors')

plt.show()

# Display metrics

print(f"Accuracy: {accuracy_knn:.4f}")

print(f"Precision: {precision_knn:.4f}")

print(f"Recall: {recall_knn:.4f}")

print(f"F1 Score: {f1_knn:.4f}")

from sklearn.metrics import roc_curve, auc

# Initialize figure

fig_roc = plt.figure(figsize=(10, 8))

models = ['Gradient Boosting Machine', 'K-Nearest Neighbors', 'Logistic Regression', 'XGBoost']

# Plot ROC curve for each model

for idx, model in enumerate([gbm_clf, knn_clf, model_lr, model_xgb]):

    if model == knn_clf:

        y_scores = model.predict_proba(X_test)

        fpr, tpr, _ = roc_curve(y_test, y_scores[:, 1], pos_label=1)

    else:

        y_scores = model.predict_proba(X_test)[:, 1]

        fpr, tpr, _ = roc_curve(y_test, y_scores, pos_label=1)

    roc_auc = auc(fpr, tpr)

    # Plot ROC curve
```

```
    plt.plot(fpr, tpr, lw=2, label=f'{models[idx]} (AUC = {roc_auc:.2f})')
# Plot ROC curve for random guessing
plt.plot([0, 1], [0, 1], linestyle='--', lw=2, color='r', label='Random Guessing')
# Set plot labels and title
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.legend(loc='lower right')
plt.grid()
# Show plot
plt.show()
import pickle
with open("Model_Prediction.sav", "wb") as f:
    pickle.dump(model_xgb,f)
with open('preprocessor.sav', 'wb') as f:
    pickle.dump(preprocessor, f)
```

CHAPTER-8: SYSTEM TESTING

System testing, also called black box testing, focuses on the external parts of the system and tests and validates the fully integrated software system. Computer systems are typically made with software integration, although the software is just a single element of any computer system. Software is usually developed in units and interfaced with other software elements and hardware, creating the complete computer system. So, computer systems consist of a set of different software applications, each designed to perform a different task, but only if they are interfaced with compatible hardware. System testing is a series of tests that examine the entire workings and behavior of the integrated software computer system against the end user's needs and requirements.

System Testing Levels

Four levels of software testing make up the hierarchy of the testing process. They are:

Unit testing. Unit testing tests a single software application.

Integration testing. Integration testing tests a group of software units.

System testing. System testing tests the entire system and is our focus for today.

Acceptance testing. Acceptance testing tests the business requirements' acceptability.

So, we see that system testing occurs after integration testing but before the acceptance testing. The testing process evaluates the end-to-end system specifications.

8.1 Types of System Testing

Although there are over 50 different types of system testing, the following seven types are the most often used.

Usability Testing. As the name implies, usability testing chiefly focuses on how easy it is for the end user to use the application, flexibility in handling controls, and the system's ability to achieve its objectives.

Load Testing. Load testing shows how a software solution will perform under real-life loads.

Regression Testing. Regression testing ensures that no changes made during development have created new bugs. Additionally, regression testing ensures no old bugs resurface after adding software modules.

Recovery Testing. Software is prone to crashing, so recovery testing is done to ascertain whether a software solution is trustworthy and can successfully recover from possible crashes.

Migration Testing. Migration testing ensures that the software can be shifted from an older system infrastructure to the current one without spawning any issues.

Functional Testing. Also called functional completeness testing, functional testing involves brainstorming any possible missing functions. For example, testers might create a list of additional functionalities to be incorporated into the application to improve it during functional testing.

Hardware/Software Testing. IBM calls Hardware/Software testing "HW/SW Testing." This testing involves focusing attention on the interactions between the hardware and software.

System Testing Process

There are two kinds of software testing: Black Box and White Box. White box testing tests a software application's internal workings or code. On the other hand, black box (or system testing) does the opposite, focusing on externals. System testing deals with the software's external workings from the user's perspective.

The system testing process is broken down into the following steps:

Test Planning. First, there needs to be a plan. The test plan is a document with all the testing information, such as strategies, objectives, entry-exit criteria, testing tools, software requirements, guidelines, etc.

Test Case Design and Execution. Next, the testers create a test case for every feature, including the test scenarios, process, description, etc. The testers then perform and execute the tests.

Defect Tracking and Defect Management. The testers then track and record any defects using the defect-tracking tools like Bugzilla, JIRA, Trello, etc. Also, the testers must document the defects so the developers can deal with and resolve them.

Reporting and Communication. Finally, the testers create bug and defect reports and send them to the developers.

8.2. Test Cases

SI.NO	Test Name	Inputs	Expected Output	Actual Output	Status
1	User Signup	Username, Full Name, Email Address, Phone	Successfully registered	Successfully registered	Pass

		Number, Password & Confirm Password			
2	User Signup	Username, Full Name, Email Address, Phone Number, Password & Confirm Password	Username already exists	Username already exists	Pass
3	Signin	Username & Password	Successfully login	Successfully login	Pass
4	Signin	Username & Password	Invalid username and password	Invalid username and password	Pass
5	Upload Input Data	Input health and lifestyle features	Data uploaded successfully	Data uploaded successfully	Pass
6	Final Outcome	Processed user input	Predicted sleep disorder status	Predicted sleep disorder status	Pass

Table-8.2.1. Test Cases

CHAPTER-9: RESULTS

9.1. Performance of Evaluation Tables

Table.9.1.1 Performance Evaluation – Engineered 5 Features

ML Model	Accuracy	Precision	Recall	F1 Score	Specificity	AUC Score	Time
Eng - Logistic Regression	0.893	0.888	0.846	0.862	0.931	0.957	0.0140
Eng - Random Forest	0.973	0.958	0.958	0.958	0.989	0.973	0.0981
Eng - Decision Tree	0.973	0.958	0.958	0.958	0.989	0.973	0.0020
Eng - SVM	0.733	0.689	0.714	0.700	0.858	0.973	0.0194
Eng - ExtraTree	0.973	0.958	0.958	0.958	0.989	0.973	0.0693
Eng - Extreme GB	0.973	0.958	0.958	0.958	0.989	0.973	0.4448
Eng - Gradient Boosting	0.973	0.958	0.958	0.958	0.989	0.973	0.1947
Eng - LightGBM	0.973	0.958	0.958	0.958	0.989	0.973	0.6603
Eng - CatBoost	0.973	0.958	0.958	0.958	0.989	0.973	1.4042
Eng - AdaBoost	0.973	0.958	0.958	0.958	0.989	0.973	0.0585

Eng - Naive Bayes	0.773	0.555	0.646	0.594	0.828	0.973	0.0015
Eng - Voting Classifier	0.973	0.958	0.958	0.958	0.989	0.973	0.2368
Eng - Stacking Classifier	0.947	0.924	0.943	0.932	0.977	0.973	0.7397
Eng - GridSearch CV	0.947	0.931	0.930	0.930	0.973	0.973	7.5317

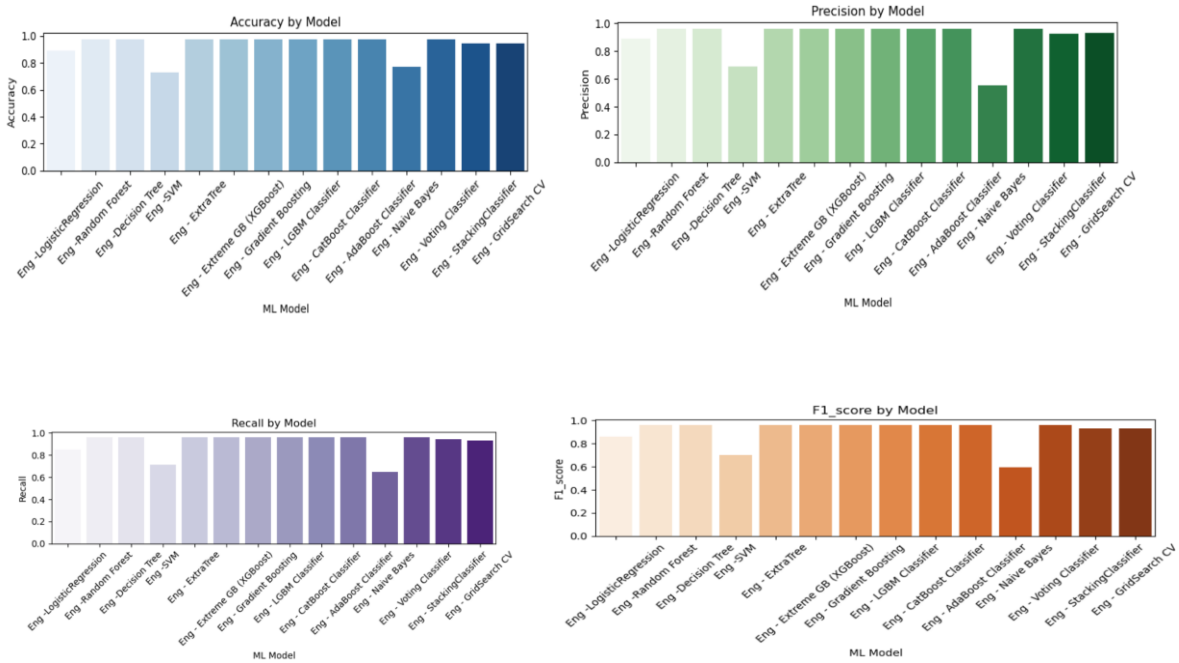
Table.9.1.2 Performance Evaluation – Engineered 2 Features

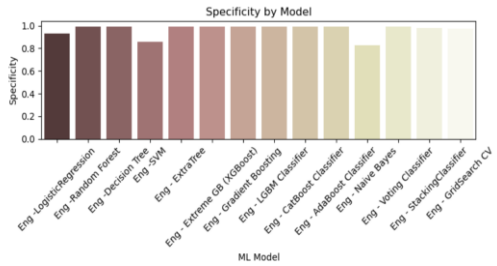
ML Model	Accuracy	Precision	Recall	F1 Score	Specificity	AUC Score	Time
Eng - Logistic Regression	0.973	0.958	0.958	0.958	0.989	0.958	0.0020
Eng - Random Forest	0.973	0.958	0.958	0.958	0.989	0.974	0.0546
Eng - Decision Tree	0.973	0.958	0.958	0.958	0.989	0.974	0.0012
Eng - SVM	0.973	0.958	0.958	0.958	0.989	0.974	0.0025
Eng - ExtraTree	0.973	0.958	0.958	0.958	0.989	0.974	0.0358
Eng - Extreme GB	0.973	0.958	0.958	0.958	0.989	0.974	0.2303
Eng - Gradient Boosting	0.973	0.958	0.958	0.958	0.989	0.974	0.0545
Eng - LightGBM	0.973	0.958	0.958	0.958	0.989	0.974	0.0842

Eng - CatBoost	0.973	0.958	0.958	0.958	0.989	0.974	0.4249
Eng - AdaBoost	0.973	0.958	0.958	0.958	0.989	0.974	0.0314
Eng - Naive Bayes	0.773	0.555	0.646	0.594	0.828	0.974	0.0010
Eng - Voting Classifier	0.973	0.958	0.958	0.958	0.989	0.974	0.1003
Eng - Stacking Classifier	0.973	0.958	0.958	0.958	0.989	0.974	0.5799
Eng - GridSearch CV	0.973	0.958	0.958	0.958	0.989	0.974	5.4550

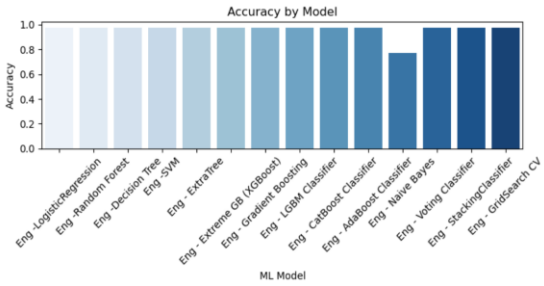
9.2 COMPARISON GRAPHS

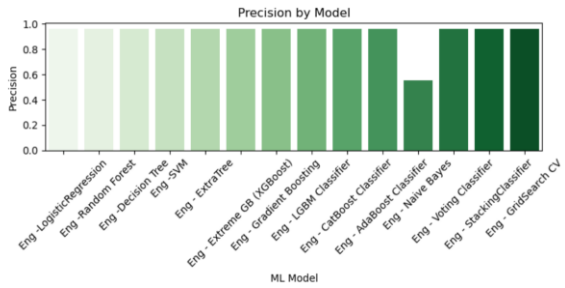
Comparison Graph – Engineered 5 Features

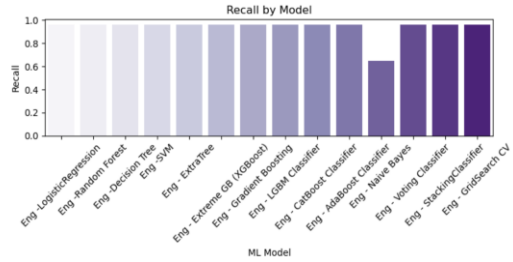


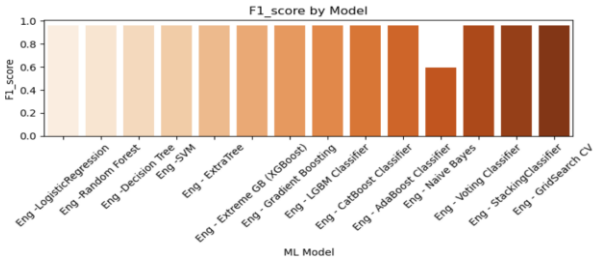


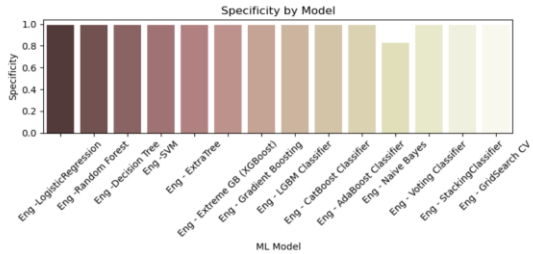
Comparison Graph – Engineered 2 Features











9.3. Outputs

Anaconda Prompt - python app.py

```
(base) C:\Users\hi>cd C:\Users\hi\Desktop\2. Improving Sleep Disorder Diagnosis Through Optimized Machine Learning Approaches

(base) C:\Users\hi\Desktop\2. Improving Sleep Disorder Diagnosis Through Optimized Machine Learning Approaches>python app.py
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
```

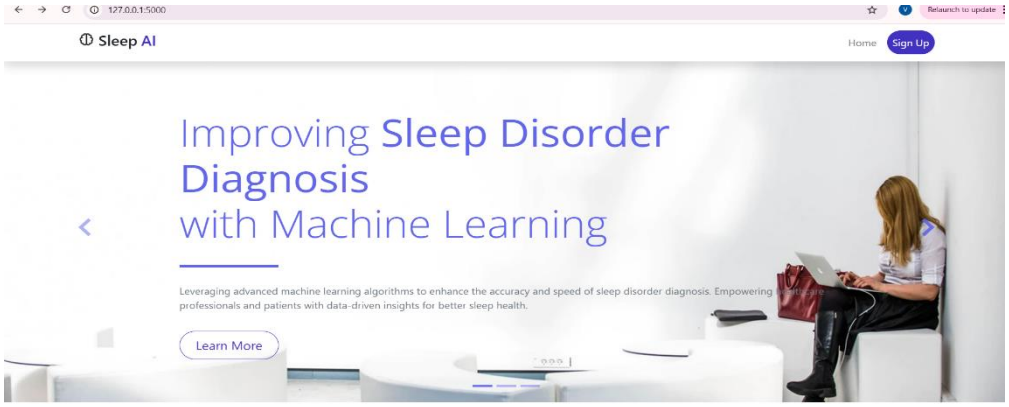


Fig.9.3.1. Homepage Interface of Sleep Disorder Prediction System

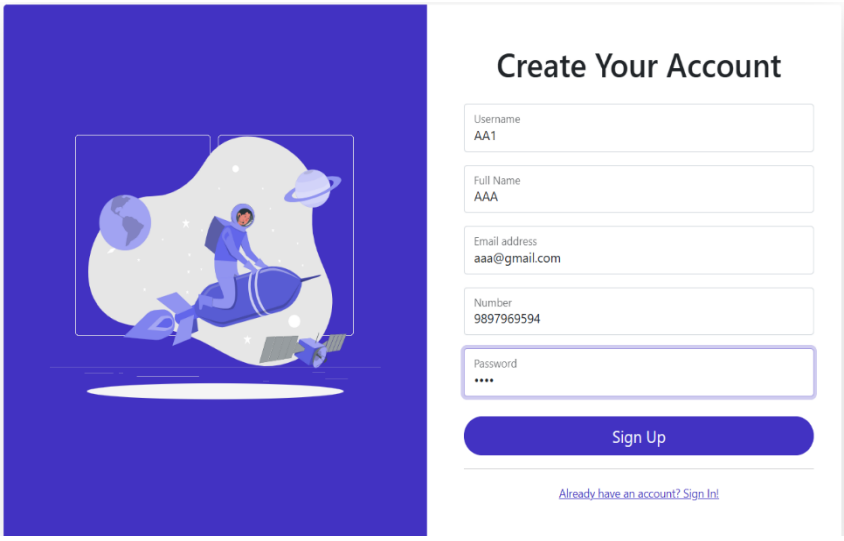


Fig.9.3.2. Account Creation Module for Sleep Disorder Prediction Application

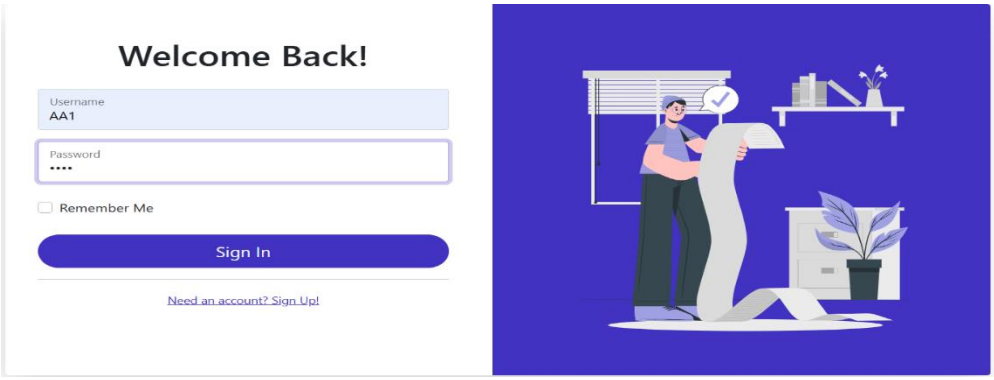


Fig.9.3.3 User Sign-In Interface for System Access

Engineering 5 Features:

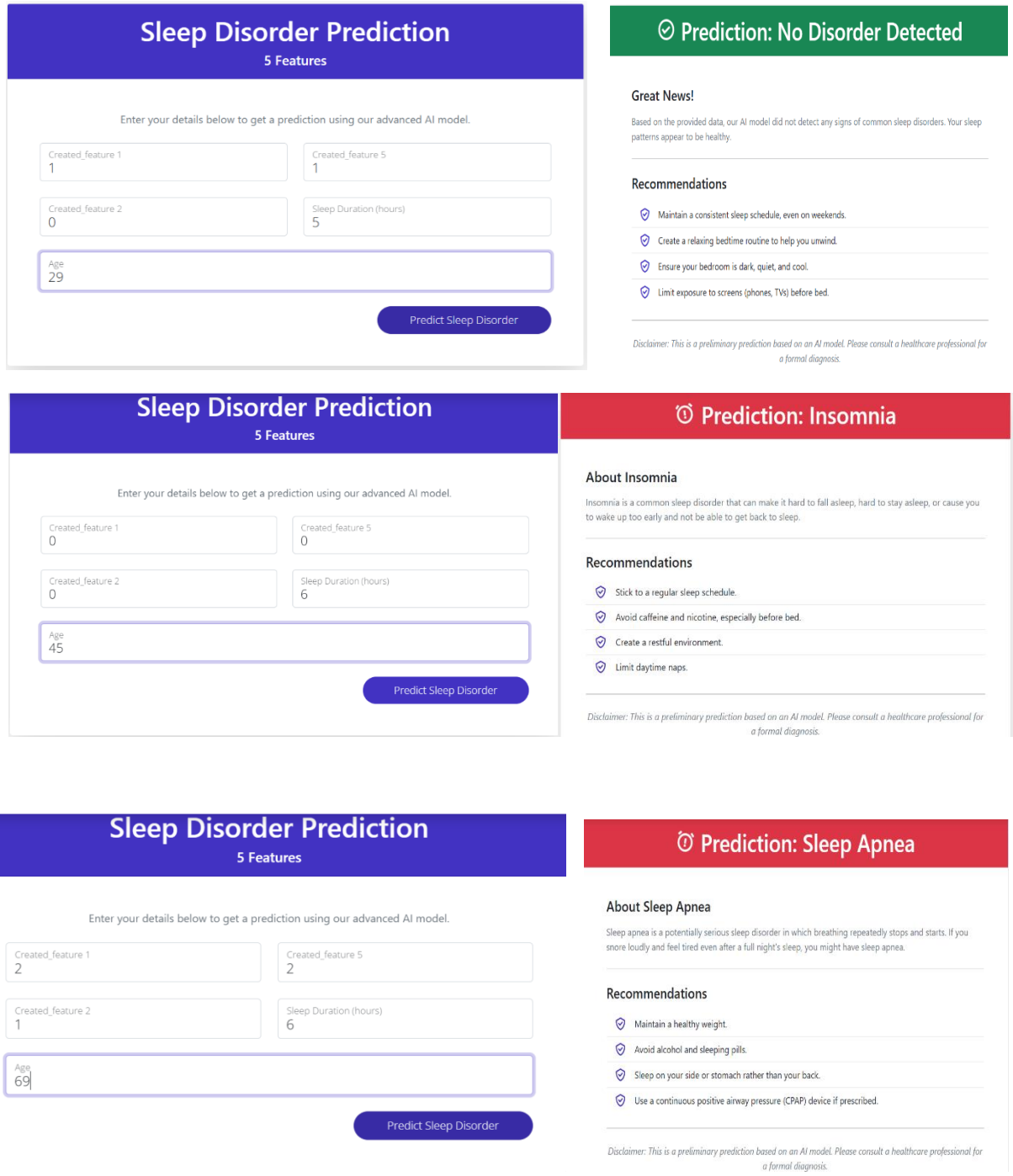


Fig.9.3.4. Optimized Five-Feature Engineered Model Output Sleep Disorder Diagnosis

Engineering 2 Features

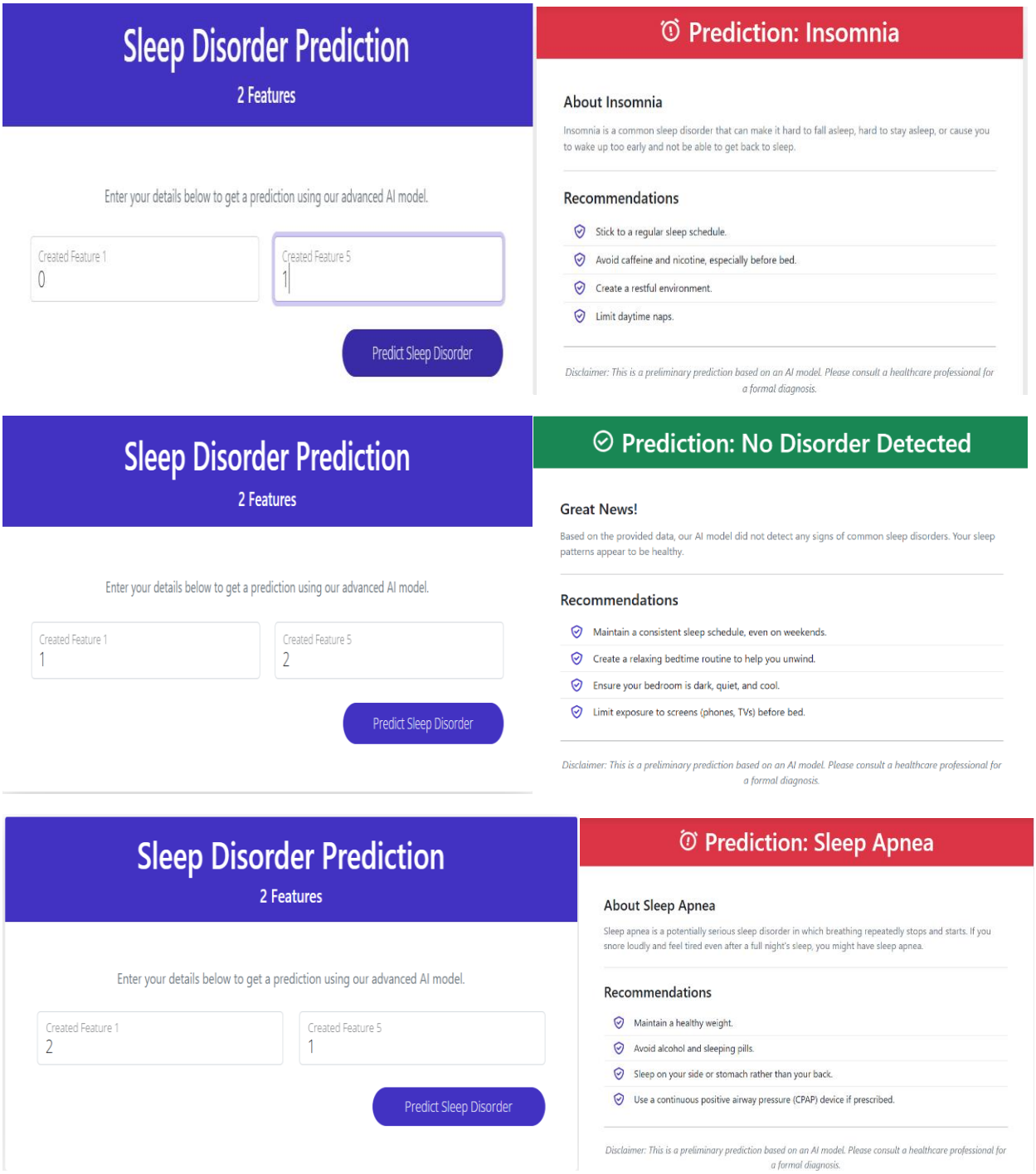


Fig.9.3.5. Reduced Feature(2-Feature) Model Performance Output of Sleep Disorder Diagnosis

CHAPTER-10: CONCLUSION AND FUTURE SCOPE

10.1. Conclusion

Accurate prediction of sleep disorders was achieved through the integration of engineered features, robust machine learning algorithms, and advanced resampling techniques. Feature selection using ANOVA and class balancing with SMOTEENN significantly enhanced the model's ability to distinguish between normal and abnormal sleep patterns. Incorporating seven engineered features derived from Random Forest, Gradient Boosting, Gaussian Naive Bayes, K-Nearest Neighbors, Decision Tree, Logistic Regression, and Support Vector Machine enabled the creation of informative representations that improved overall predictive performance. Among the tested models, the Voting Classifier achieved the highest accuracy of 97.3% for both five- and two-feature engineered datasets, demonstrating its robustness and reliability in sleep disorder classification. When using the original features, the Stacking Classifier achieved accuracies of 88.0% with five features and 80.0% with two features, confirming that engineered features substantially enhance model performance. Explainable AI techniques, including LIME and SHAP, provided interpretability by highlighting the most influential features contributing to model predictions, supporting transparency and clinical relevance. These results indicate that optimized machine learning models, coupled with engineered features and XAI methods, can reliably assist in accurate and interpretable sleep disorder diagnosis.

10.2. Future Scope

Future research can expand this study by incorporating larger and more diverse sleep-related datasets that include physiological signals such as EEG, ECG, and respiratory measures to enhance model generalization. Deep learning approaches, including Convolutional Neural Networks and Recurrent Neural Networks, could be explored for automated feature learning, potentially capturing complex temporal dependencies in sleep data. Integration of multimodal data combining lifestyle, medical history, and wearable device outputs can further improve diagnostic accuracy. Optimization techniques such as Bayesian hyperparameter tuning and advanced ensemble strategies may refine model performance beyond current results. Additionally, real-time deployment frameworks can be developed to provide continuous monitoring, enabling practical applications of machine learning models in healthcare systems for proactive sleep disorder detection.

CHAPTER-11: REFERENCES

[1] Khanmohmmadi, S., Khatibi, T., Tajeddin, G., Akhondzadeh, E., & Shojaee, A. (2025). Revolutionizing sleep disorder diagnosis: A Multi-Task learning approach optimized with genetic and Q-Learning techniques. *Scientific Reports*, *15*(1), 16603.

[2] Z. Zhang, T. B. Conroy, A. C. Krieger, and E. C. Kan, "Detection and prediction of sleep disorders by covert bed-integrated RF sensors," IEEE Trans. Biomed. Eng., vol. 70, no. 4, pp. 1208–1218, Apr. 2023.

[3] Chandra, S. O., Jenefa, A., Thiyagu, T. M., & Vidhya, K. (2025, June). Enhancing Diagnostic Precision: Advanced Neural Networks for Sleep Disorder Classification. In *2025 3rd International Conference on Self Sustainable Artificial Intelligence Systems (ICSSAS)* (pp. 1097-1102). IEEE.

[4] Frantzidis, C. A. (2025). From Narratives to Diagnosis: A Machine Learning Framework for Classifying Sleep Disorders in Aging Populations: The sleepCare Platform. *Brain Sciences*, *15*(7), 667.

[5] Panda, N. R., Paramanik, S., Raut, P. K., & Bhuyan, R. (2025). Prediction of sleep disorders using Novel decision support neutrosophic based machine learning models. *Neutrosophic Sets and Systems*, *82*(1), 19.

[6] T. S. Alshammari, "Applying machine learning algorithms for the classification of sleep disorders," IEEE Access, vol. 12, pp. 36110–36121, 2024.

[7] X. Xiao, Y. Rui, Y. Jin, and M. Chen, "Relationship of sleep disorder with neurodegenerative and psychiatric diseases: An updated review," Neurochemical Res., vol. 49, no. 3, pp. 568–582, Mar. 2024.

[8] M. Braun, S. Dietz-Terjung, U. Sommer, C. Schoebel, and C. Heiser, ‘‘Stated patient preferences for overnight at-home diagnostic assessment of sleep disorders,’’ Sleep Breathing,vol.28,no.5,pp. 1939–1949,Oct.2024.

[9]A.Şenol,T.Talan,andC.Aktürk,‘‘Anewhybridfeaturereductionmethod by using MCMSTClustering algorithm with various feature projection methods: A case study on sleep disorder diagnosis,’’ Signal, Image Video Process., vol. 18, no. 5, pp. 4589–4603, Jul. 2024.

[10] I. A. Hidayat, ‘‘Classification of sleep disorders using random forest on sleep health and lifestyle dataset,’’ J. Dinda: Data Sci., Inf. Technol., Data Anal., vol. 3, no. 2, pp. 71–76, Aug. 2023.